UNIT I – Introduction to AI & Problem-Solving Agents

AI & ML for Mechanical Engineering (23A03701)

1. What is Artificial Intelligence?

➤ Definition

AI = the science and engineering of making machines (computers, robots, software) do things that would need intelligence if done by humans – learning, reasoning, perceiving, planning, understanding language.

The term was coined by **John McCarthy** in **1956** (Dartmouth Conference).

➤ Four views of AI (Russell & Norvig)

	Human-like	Rational (ideal)
Thinking	Thinking humanly – cognitive modelling	Thinking rationally – "laws of thought", logic
Acting	Acting humanly – **Turing Test**	Acting rationally – **rational agent** (most used)

Turing Test (1950): a human interrogator asks written questions; if he cannot tell whether the answers come from a human or a machine, the machine is said to be intelligent.

➤ Why AI for Mechanical Engineers?

- Predictive maintenance of pumps, turbines, bearings
- Robotics & autonomous vehicles, path planning
- Design optimisation, generative design, CFD / FEA surrogates
- Smart manufacturing (Industry 4.0), quality inspection by vision
- Process control – fuzzy / adaptive controllers

2. Problems of AI (areas / task domains)

Category	Tasks
Mundane tasks	Perception (vision, speech), natural language understanding & generation, commonsense reasoning, robot control
Formal tasks	Games (chess, checkers), mathematics (geometry, logic, theorem proving), integral calculus
Expert tasks	Engineering design, scientific analysis, medical diagnosis, financial analysis, fault finding

- Humans do *mundane* tasks easily but computers find them hard; *formal* tasks look hard but are easier to program.
- Typical AI problems need **search**, **knowledge** and **abstraction**.

➤ Key difficulties

- Huge search spaces (combinatorial explosion)
- Knowledge is vast, incomplete, uncertain and changing
- Real world is noisy – sensors give partial information
- Real-time response needed (robots, vehicles)

3. The AI Technique

An **AI technique** is a method that exploits knowledge in a way that makes the problem easier to solve.

➤ Properties of knowledge

- It is **voluminous** (huge)
- It is **hard to characterise accurately**
- It is **constantly changing**
- It differs from data – it is **organised** so it can be **generalised** (captures regularities) and **understood** by the people who provide it
- It can be **easily modified** to correct errors and reflect changes

- It can be used in many situations even if incomplete – helps ***narrow*** the possibilities

Three important AI techniques:

- ***Search*** – provides a way of solving problems for which no direct approach is available (try alternatives systematically).
- ***Use of knowledge*** – solve complex problems by exploiting the structure of objects involved.
- ***Abstraction*** – separate important features from unimportant details so that the process is not overwhelmed.

➤ Underlying assumption (Physical Symbol System Hypothesis – Newell & Simon)

"A physical symbol system has the necessary and sufficient means for general intelligent action."

4. Tic-Tac-Toe Problem – three approaches

Aim: show how a ***better data structure + more knowledge*** make a program smarter. Board = 3×3, players X and O, 3 in a line wins.

➤ Program 1 – Lookup table (simple)

- Board = vector of 9 elements (0 = blank, 1 = X, 2 = O). Treat as a ***ternary number***.
- Convert to decimal → index into a table of 3^9 = 19683 entries; each entry stores the ***next board (move)***.

Drawbacks: huge memory, someone must fill all entries by hand, not extendable (3-D tic-tac-toe needs 3^{27} entries), not intelligent.

➤ Program 2 – Magic-square / turn-based

- Board = 9-element vector, but 2 = blank, 3 = X, 5 = O. Turn counter 1…9.
- Functions: ***Make2*** (play centre if free, else a non-corner), ***Posswin(p)*** (returns the square that wins for p, else 0 – product of a line = 18 for X, 50 for O), ***Go(n)*** (play in square n).
- Strategy by turn: odd turns X plays, even turns O. Win if possible, else block, else take centre, else corner.

Advantages: less space, strategy can be explained. Disadvantage: it is ***not flexible*** – cannot be generalised to other games.

➤ Program 3 – Game tree / minimax style

- Board position = 9-element vector plus a ***list of positions reachable*** by one legal move, plus a ***rating*** (how likely to lead to a win).
- Look ahead: generate all successors, evaluate each, back up values; choose the move with best rating.

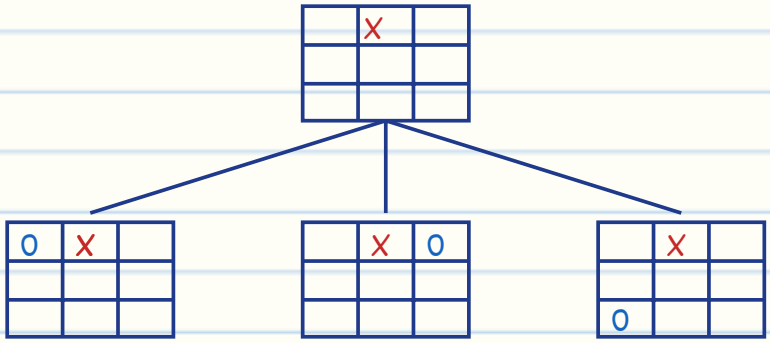


Fig: a partial game tree for Tic-Tac-Toe (X to move, O replies)

Program 3 is ***slower*** but more ***general*** and lets us use ***look-ahead*** – the same idea is used in chess programs. It shows that more knowledge + better structure → better program.

5. Intelligent Agents

➤ Agent

An ***agent*** is anything that perceives its environment through sensors and acts upon it through actuators.

- ***Human agent:*** eyes, ears (sensors); hands, legs (actuators)
- ***Robotic agent:*** cameras, IR, encoders (sensors); motors, grippers (actuators)
- ***Software agent:*** keystrokes, files, packets (sensors); screen display, files (actuators)

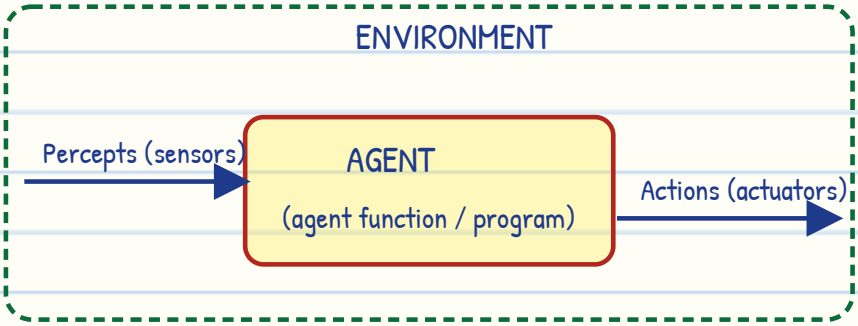


Fig: Agent interacting with environment

- ***Percept*** = agent's perceptual input at one instant; ***percept sequence*** = complete history of percepts.
- ***Agent function*** maps percept sequence → action. ***Agent program*** = concrete implementation running on the ***architecture*** (hardware). Agent = architecture + program.

➤ Rational agent

For each possible percept sequence, a rational agent selects the action expected to ***maximise its performance measure***, given the evidence from percepts and built-in knowledge.

Rationality depends on: (1) performance measure, (2) prior knowledge of environment, (3) actions it can perform, (4) percept sequence to date.

> ***Example – Vacuum-cleaner agent:*** world = squares A, B; percepts = [location, Clean/Dirty]; actions = Left, Right, Suck. Rule: if Dirty → Suck; else move to the other square. Performance measure: +1 per clean square per time-step.

Omniscience ≠ rationality. A rational agent maximises *expected* performance; it also needs ***information gathering***, ***learning*** and ***autonomy***.

6. Nature of Environments – PEAS

To design an agent we first specify the ***task environment*** as PEAS: Performance measure, Environment, Actuators, Sensors.

Agent	P	E	A	S
Automated car / taxi	Safe, fast, legal, comfortable, profit	Roads, traffic, pedestrians	Steering, accelerator,	Cameras, LIDAR, GPS, speedometer

			brake, horn	
Industrial robot (pick & place)	% parts correctly placed, cycle time	Conveyor belt, parts	Jointed arm, gripper	Camera, joint-angle sensors, force sensor
Medical / fault diagnosis system	Correct diagnosis, minimal cost	Patient / machine, staff	Display of tests, diagnosis	Keyboard entry of symptoms, vibration data

➤ Properties of task environments

Property	Meaning (easy ⟷ hard)
Fully ⟷ Partially observable	Sensors give complete state at each time? (chess – full; taxi – partial)
Deterministic ⟷ Stochastic	Next state fully determined by current state + action? (chess – det.; taxi – stochastic). *Strategic* = deterministic except other agents' actions.
Episodic ⟷ Sequential	Are experiences divided into independent episodes? (part inspection – episodic; chess – sequential)
Static ⟷ Dynamic	Does environment change while agent deliberates? (crossword – static; taxi – dynamic). *Semi-dynamic*: only the score changes with time (timed chess).
Discrete ⟷ Continuous	Finite number of states/actions? (chess – discrete; robot arm – continuous)
Single ⟷ Multi-agent	One agent or several (competitive / cooperative)?
Known ⟷ Unknown	Does agent know the "laws" of the environment?

Hardest case: partially observable, stochastic, sequential, dynamic, continuous, multi-agent (e.g. driving).

7. Structure of Agents (types)

➤ (a) Simple reflex agent

- Acts only on the ***current percept*** using ***condition–action rules*** (if … then …). No memory.
- Example: thermostat, "if car in front is braking → brake", automatic door.
- Works only if environment is ***fully observable***; may loop forever in partial observability.

```
function SIMPLE-REFLEX-AGENT(percept)
  state  ← INTERPRET-INPUT(percept)
  rule   ← RULE-MATCH(state, rules)
  return rule.ACTION
```

➤ (b) Model-based reflex agent

- Keeps an ***internal state*** (model) of the part of the world it cannot see – "how the world evolves" + "what my actions do".
- Example: car remembers that the vehicle behind exists although it is not in the camera now.

➤ (c) Goal-based agent

- Has an explicit ***goal*** (desirable situation). Chooses actions that achieve the goal using ***search and planning*** – "what will happen if I do A?"
- More ***flexible***: changing the goal changes behaviour without rewriting rules. Example: route-finding robot, taxi going to a destination.

➤ (d) Utility-based agent

- Goals give only a yes/no (happy / unhappy). ***Utility function*** maps a state to a real number = degree of happiness.
- Chooses the action with ***maximum expected utility***; handles conflicting goals (speed vs safety) and uncertainty. Example: choose the route that is fast *and* safe *and* cheap.

➤ (e) Learning agent

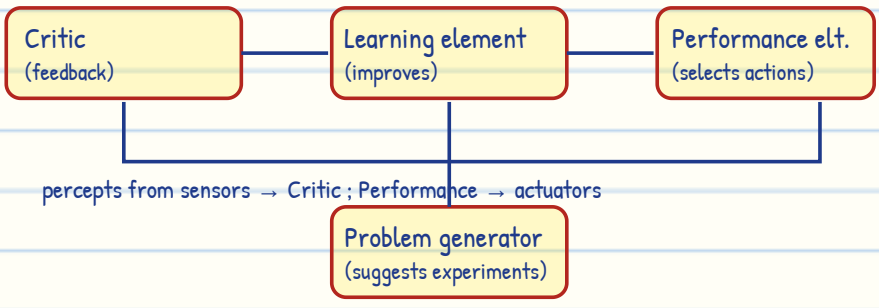


Fig: General learning agent

- ***Performance element*** – selects external actions (the whole agent of earlier types).
- ***Learning element*** – makes improvements using knowledge from the critic.
- ***Critic*** – tells how well the agent is doing vs a fixed performance standard.

- **Problem generator** – suggests exploratory actions leading to new experiences.

➤ Quick comparison

Agent	Uses	Memory?	Looks ahead?
Simple reflex	Condition-action rules	No	No
Model-based	Internal state + rules	Yes	No
Goal-based	Goal + search	Yes	Yes
Utility-based	Utility function	Yes	Yes (best action)
Learning	Feedback + critic	Yes	Improves with experience

8. Defining the Problem as State Space Search

➤ State space

The set of **all possible states** reachable from the start state by applying legal **operators (rules)**. A problem is solved by finding a path from the initial state to a goal state.

➤ Steps to define a problem

1. **Define the state space** – initial state, possible states.
2. **Specify initial state(s).**
3. **Specify goal state(s)** (one or many).
4. **Specify the rules / operators** – legal actions, with their preconditions and effects; keep them general, not too detailed.

A well-defined problem has: **initial state, actions (successor function), goal test, path cost**. A solution with the lowest path cost is **optimal**.

> **Example 1 – Water-jug problem:** 4-litre jug (x) and 3-litre jug (y), no markings, pump available. Goal: exactly 2 litres in the 4-litre jug.
> State = (x, y), start (0,0), goal (2, n). Rules include: fill 4-jug (x,y) → (4,y); fill 3-jug → (x,3); empty either; pour 3-jug into 4-jug until full or empty: (x,y) → (x+y,0) if x+y≤4; and so on.
> **Solution path:** (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0) ✔

Example 2 – 8-puzzle: 3×3 board, 8 tiles + blank. State = tile positions; operators = move blank Up/Down/Left/Right; goal = ordered arrangement; path cost = 1 per move.

Example 3 – Missionaries & Cannibals: 3 + 3 on a bank, boat takes 2. Cannibals must never outnumber missionaries on either bank. State = (M, C, boat side).

Example 4 – Travelling Salesman / robot path / assembly sequence – same idea: states = partial tours/placements.

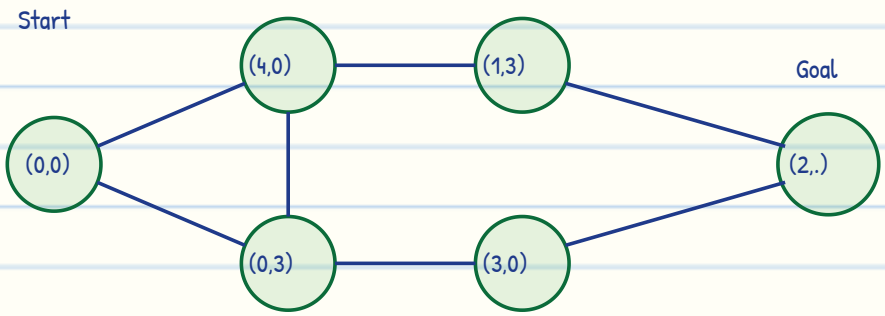


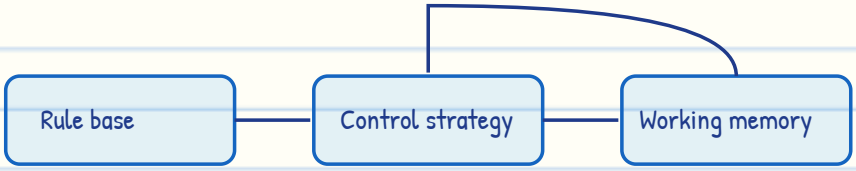
Fig: part of the water-jug state space (nodes = states, edges = operators)

9. Production Systems

➤ Definition

A ***production system*** consists of:

1. ***A set of production rules*** – each is ***condition → action*** (left side = pattern; right side = operation).
2. ***One or more knowledge bases / databases*** – current facts (working memory) and global state.
3. ***A control strategy*** – decides which rule to apply when many match; resolves conflicts (and avoids looping).
4. ***A rule applier*** – matches the pattern and executes the action.



match → select → apply → repeat until goal

➤ Control strategy requirements

- ***Causes motion*** – it must not stand still (must apply some rule each cycle).
- ***Systematic*** – do not repeat the same state needlessly (avoid infinite loops).

➤ Classes of production systems

- ***Monotonic*** – applying a rule never prevents a later rule that was applicable earlier from being applied.
- ***Non-monotonic*** – not monotonic.
- ***Partially commutative*** – if a sequence of rules transforms state x to y, any *permutation* of the allowable rules also transforms x to y.
- ***Commutative*** – both monotonic and partially commutative.

	Monotonic	***Non-monotonic***
Partially commutative	Theorem proving	Robot navigation
Not partially commutative	Chemical synthesis	Bridge / chess

Commutative systems are the easiest – ***ignorable*** (e.g. theorem proving) problems can be solved without backtracking; ***recoverable*** (8-puzzle) need backtracking; ***irrecoverable*** (chess) need planning/look-ahead.

10. Problem Characteristics

Before choosing a method, analyse the problem with these questions:

1. ***Is the problem decomposable?*** Can it be broken into independent sub-problems? (symbolic integration – yes; blocks world – interacting sub-goals).
2. ***Can solution steps be ignored or undone?*** *Ignorable* (theorem proving), *recoverable* (8-puzzle), *irrecoverable* (chess) – decides if backtracking/planning is needed.

3. ***Is the universe predictable?*** Certain-outcome (8-puzzle) vs uncertain-outcome (bridge, robot with noisy sensors).
4. ***Is a good solution absolute or relative?*** Any-path problem (find one proof) vs best-path problem (TSP – shortest route) – best-path searches all paths.
5. ***Is the solution a state or a path?*** Interpretation of a sentence → just a state; water-jug → the path of operations.
6. ***What is the role of knowledge?*** Chess needs little knowledge (rules) but newspaper-story understanding needs vast knowledge.
7. ***Does the task need interaction with a person?*** Solitary (give problem, get answer) vs conversational (user feedback during solving).

11. Issues in the Design of Search Programs

- ***Direction of search – forward*** (from start to goal) or ***backward*** (goal to start); choose by branching factor, whether goals/starts are more numerous, and whether justification is needed.
- ***Choosing the right rules*** – avoid too many or too specific rules; use ***indexing*** / hashing to match quickly.
- ***Representation of nodes – tree vs graph***: graphs detect repeated states (need check of OPEN/CLOSED lists), trees may re-explore the same state.
- ***Search-tree vs search-graph*** – avoiding duplicate nodes saves effort but costs checking time.
- ***Heuristic knowledge*** – use of an ***evaluation function*** to choose which node to expand next.
- ***Size of the search space*** – combinatorial explosion; ***pruning*** is essential.
- ***Matching*** – how to pick applicable rules efficiently.
- ***Termination & completeness / optimality*** – will the search always find a solution, the best one, within time and memory?

➤ Evaluating a search strategy

- ***Completeness*** – guaranteed to find a solution if one exists?
- ***Optimality*** – finds the least-cost solution?

- ***Time complexity*** – nodes generated (b = branching factor, d = depth of goal).
- ***Space complexity*** – maximum nodes stored in memory.

12. Important Exam Questions

1. Define AI. Explain the different approaches / the Turing Test.
2. What is an AI technique? List its properties.
3. Explain the tic-tac-toe problem with the three programs and compare.
4. Define agent and rational agent. Explain with PEAS (give an example).
5. Explain the types of agent programs with block diagrams.
6. Explain the properties of task environments.
7. Formulate (a) water-jug, (b) 8-puzzle as state-space search problems.
8. What is a production system? Explain its components and categories.
9. Discuss the characteristics of a problem.
10. Discuss the issues in the design of search programs.

✎ *All the best! Revise definitions, diagrams and examples.*

UNIT II – Search Techniques

AI & ML for Mechanical Engineering (23A03701)

1. Problem-Solving Agents

A ***problem-solving agent*** is a goal-based agent that decides what to do by finding a ***sequence of actions*** that leads to a desired goal state.

➤ Steps followed by the agent

- ***Goal formulation*** – decide which states are "goal states" (e.g. reach the workshop).
- ***Problem formulation*** – decide which actions & states to consider.
- ***Search*** – look for the sequence of actions that reaches the goal.
- ***Execution*** – carry out the actions of the solution one by one.

Cycle: ***Formulate*** → ***Search*** → ***Execute*** (the agent ignores percepts while executing – "open-loop" system).

➤ Well-defined problem has 5 parts

- ***Initial state*** – where the agent starts, e.g. In(Arad).
- ***Actions*** – set of possible actions, e.g. Go(Sibiu).
- ***Transition model*** – RESULT(s,a) = new state.
- ***Goal test*** – is the current state a goal?
- ***Path cost*** – numeric cost of a path = sum of step costs c(s,a,s').

Solution = path from initial state to goal state. ***Optimal solution*** = the one with lowest path cost.

> ***M.E. example:*** Robot arm in a welding shop must move from home position to a weld point avoiding fixtures. State = (joint angles), action = rotate a joint by a small step, goal = tool-tip at weld point, path cost = total joint rotation / time.

2. Searching for Solutions

The possible action sequences form a ***search tree***: root = initial state, branches = actions, nodes = states.

- ***Expanding*** a node = applying all legal actions, generating child nodes.
- ***Frontier (open list)*** = set of generated nodes not yet expanded.
- ***Explored set (closed list)*** = nodes already expanded – avoids repeated states / loops.
- ***Search strategy*** = rule for choosing which frontier node to expand next.

A ***node*** stores: STATE, PARENT, ACTION, PATH-COST g(n).

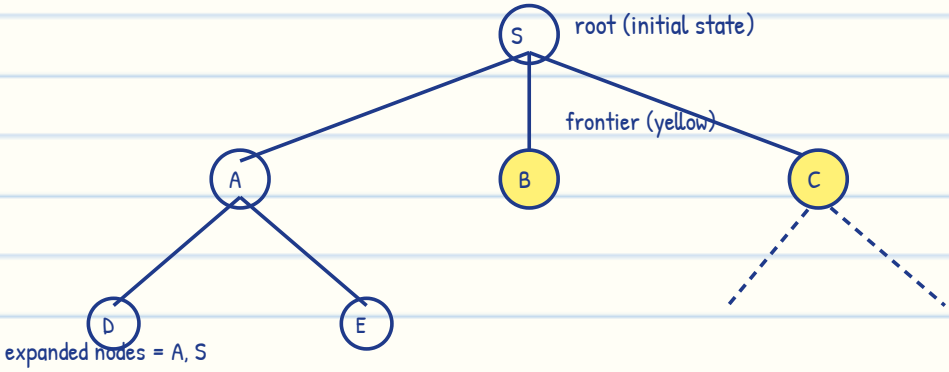


Fig 2.1 – A search tree: S expanded, A expanded, B & C in frontier

➤ Tree search vs Graph search

Tree search	Graph search
No explored set; may revisit same state many times	Keeps explored set; each state expanded once
Can loop forever on cycles	Avoids loops & redundant paths
Less memory	More memory

➤ Measuring problem-solving performance

- ***Completeness*** – is a solution guaranteed if one exists?
- ***Optimality*** – does it find the least-cost solution?
- ***Time complexity*** – number of nodes generated.
- ***Space complexity*** – max nodes stored in memory.

Complexity is expressed with: ***b*** = branching factor (max children), ***d*** = depth of shallowest goal, ***m*** = max depth of state space, ***l*** = depth limit.

3. Uninformed (Uniform / Blind) Search Strategies

These strategies have no extra information about states beyond the problem definition – they only generate successors and test for goal. Also called ***blind search***.

Types: BFS, Uniform-cost search, DFS, Depth-limited, Iterative deepening, Bidirectional.

➤ 3.1 Breadth-First Search (BFS)

Expand the ***shallowest*** unexpanded node first. Frontier is a ***FIFO queue***.

➤ Algorithm

1. Put the start node in a FIFO queue; explored = { }.
2. If queue empty → FAILURE.
3. Remove first node n. If n is goal → return solution.
4. Expand n; add children not in explored/frontier to the END of queue.
5. Go to step 2.

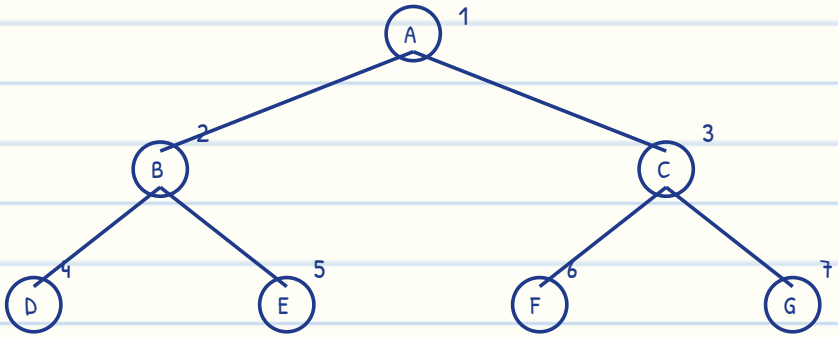


Fig 3.1 – BFS order: A B C D E F G (level by level)

- ***Complete:*** Yes (if b is finite).
- ***Optimal:*** Yes, only when all step costs are equal.
- ***Time:*** $O(b^d)$ ***Space:*** $O(b^d)$ – keeps every node in memory (big drawback).

Example: b = 10, d = 6 → 10^6 nodes; at 1 KB/node ≈ 1 GB. Memory runs out before time does!

Uniform-cost search (UCS): BFS variant that expands the node with the ***lowest path cost g(n)*** (priority queue). Optimal for any positive step cost. Complexity $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$, C^* = optimal cost.

➤ 3.2 Depth-First Search (DFS)

Expand the ***deepest*** unexpanded node first. Frontier is a ***LIFO stack*** (or recursion). Backtracks when a dead-end is reached.

➤ Algorithm

1. Push start node on stack.
2. Pop top node n. If goal → stop.
3. Push its children (left to right) on the stack.
4. Repeat; if stack empty → FAILURE.

For the tree of Fig 3.1, DFS order: **A** → **B** → **D** → **E** → **C** → **F** → **G**.

- ***Complete:*** No – fails in infinite-depth spaces or loops (tree search). Complete in finite graph with explored set.
- ***Optimal:*** No – may return a deeper, costlier solution first.
- ***Time:*** $O(b^m)$ – can be terrible if $m \gg d$.
- ***Space:*** $O(bm)$ – linear, the big advantage.

3.3 Depth-Limited Search (DLS)

DFS with a predetermined ***depth limit l***. Nodes at depth l are treated as if they have no successors → solves the infinite-path problem.

- Returns ***failure*** (no solution) or ***cutoff*** (no solution within limit l).
- ***Complete:*** No if $l < d$ (goal beyond limit). ***Optimal:*** No if $l > d$.
- ***Time:*** $O(b^l)$ ***Space:*** $O(bl)$.
- Limit can come from domain knowledge – e.g. a map with 20 cities → any solution has length ≤ 19, so l = 19 (diameter of the graph gives a better limit).

```
function DLS(node, goal, limit):
  if node == goal: return SOLUTION
  if limit == 0:   return CUTOFF
  for each child of node:
      r = DLS(child, goal, limit-1)
      if r == SOLUTION: return r
  return FAILURE / CUTOFF
```

➤ 3.4 Iterative Deepening DFS (IDDFS)

Repeatedly run DLS with l = 0, 1, 2, 3 … until goal is found. Combines ***BFS completeness/optimality*** with ***DFS memory***.

- Complete: Yes. Optimal: Yes (unit step cost). Time $O(b^d)$. Space O(bd).
- Repeated work is small: most nodes are at the bottom level. Nodes generated = $(d)b + (d-1)b^2 + \dots + b^d$.
- Preferred uninformed method when search space is large and depth of solution unknown.

➤ 3.5 Bidirectional Search

Run ***two searches simultaneously***: one ***forward*** from the initial state, one ***backward*** from the goal; stop when the two frontiers ***meet***.

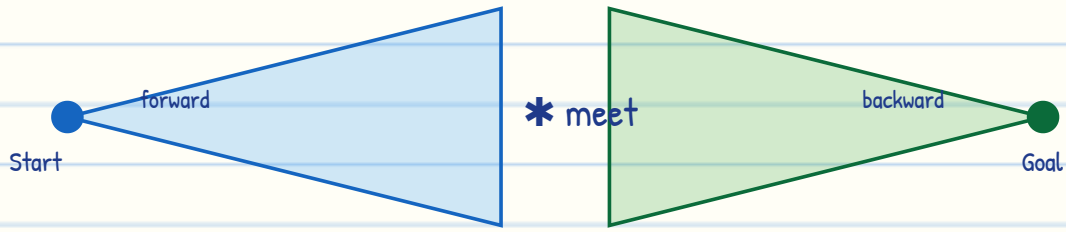


Fig 3.2 – Two small cones ($b^{d/2}$ each) instead of one big cone (b^d)

- ***Time:*** $O(b^{d/2})$ ***Space:*** $O(b^{d/2})$. Example: b=10, d=6 → 2×10^3 = 2000 nodes vs 10^6.
- ***Complete:*** Yes (BFS both sides). ***Optimal:*** Yes (unit costs, BFS both sides).
- ***Requirements / drawbacks:*** goal state must be explicitly known and unique; predecessors must be computable (reversible actions); need an efficient check that frontiers intersect; high memory.

➤ 3.6 Comparing Uninformed Strategies

Criterion	BFS	UCS	DFS	DLS	IDDFS	Bidirectional
Complete?	Yes	Yes	No	No	Yes	Yes
Time	b^d	$b^{1+C^*/\varepsilon}$	b^m	b^l	b^d	$b^{d/2}$
Space	b^d	$b^{1+C^*/\varepsilon}$	bm	bl	bd	$b^{d/2}$
Optimal?	Yes*	Yes	No	No	Yes*	Yes*

** when all step costs are equal. b = branching factor, d = goal depth, m = max depth, l = limit.*

4. Heuristic (Informed) Search Strategies

Informed search uses ***problem-specific knowledge*** beyond the definition, through an ***evaluation function f(n)***, to find solutions more efficiently than blind search.

➤ Heuristic function h(n)

h(n) = ***estimated cost of the cheapest path from node n to a goal***. h(goal) = 0.

- ***Admissible:*** $h(n) \leq h^*(n)$ (never overestimates the true cost).
- ***Consistent (monotonic):*** $h(n) \leq c(n,a,n') + h(n')$.
- Examples: straight-line distance (SLD) in route finding; Manhattan distance in 8-puzzle; number of misplaced tiles.

Notation: ***g(n)*** = cost from start to n, ***h(n)*** = estimated cost n to goal, ***f(n)*** = evaluation function.

➤ 4.1 Greedy Best-First Search

Expands the node that ***appears closest to the goal***: f(n) = h(n). Frontier = priority queue ordered by h.

- ***Complete:*** No (can get stuck in loops, tree version). ***Optimal:*** No.
- ***Time & Space:*** $O(b^m)$ worst case; a good heuristic gives big improvement.
- Advantage: fast, simple. Disadvantage: ignores cost already spent g(n), so can follow a "cheap-looking" but long path.

Example (route): S → A (cost 1), S → B (cost 4). h(A)=7, h(B)=2, h(G)=0. B → G cost 3; A → G cost 9. Greedy picks B (h smaller) then G: path S-B-G, cost 7. Here also optimal, but in general not guaranteed.

➤ 4.2 A* Search

Most popular informed search. Combines cost so far and estimated cost to go:

$$f(n) = g(n) + h(n)$$

➤ Algorithm

1. OPEN = {start}, CLOSED = { }; f(start) = h(start).
2. If OPEN empty → FAILURE.
3. Pick node n in OPEN with ***lowest f(n)***; move it to CLOSED.
4. If n is goal → return path (follow parents).
5. For each successor m: compute g(m) = g(n)+c(n,m), f(m) = g(m)+h(m). If m is new → add to OPEN; if m already present with higher g → update parent & cost.
6. Go to step 2.

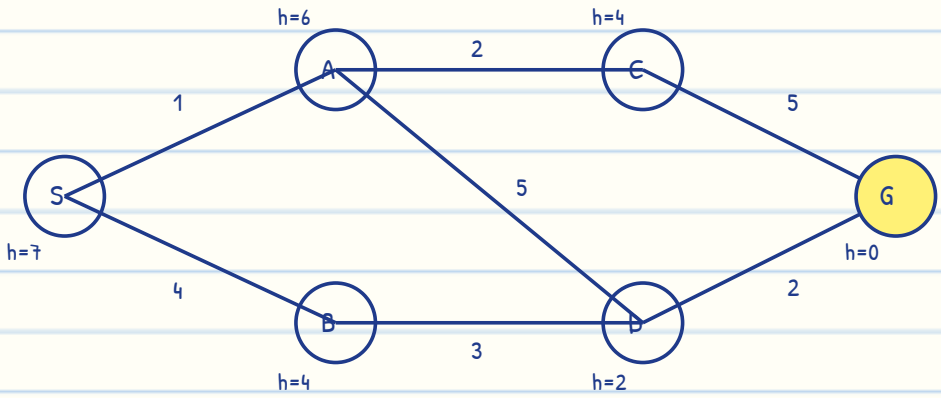


Fig 4.1 – Edge numbers = step cost, red = h(n)

➤ Trace

Step	Expand	OPEN (node : f = g+h)
1	S	A:1+6=7, B:4+4=8
2	A (f=7)	B:8, C:3+4=7, D:6+2=8 (via A)
3	C (f=7)	B:8, D:8, G:8+0=8
4	B (f=8)	D: via B = 4+3+2 = 9 (worse, keep 6+2=8), G:8
5	D (f=8)	G: 6+2=8 → better path via D: g=8
6	G	goal reached, cost = 8

Optimal path: **S** → **A** → **D** → **G** (1+5+2 = 8). Same cost as S-A-C-G (1+2+5=8).

- ***Complete:*** Yes. ***Optimal:*** Yes if h is admissible (tree search) / consistent (graph search).
- ***Time:*** exponential in (h error × solution length). ***Space:*** $O(b^d)$ – keeps all nodes; main drawback.
- A* is ***optimally efficient*** – no other optimal algorithm using same h expands fewer nodes.

Greedy BFS	A*
f = h	f = g + h
Not optimal, not complete	Optimal & complete (admissible h)
Faster, less memory in practice	Slower, heavy memory

4.3 AO* Search (AND-OR graphs)

Used for problems that can be ***decomposed into sub-problems***; solved by an ***AND-OR graph***.

- ***OR node:*** solved if any one child is solved (alternatives).
- ***AND node (AND-arc):*** solved only if all children are solved.
- Solution is not a path but a ***sub-graph (solution graph)***.

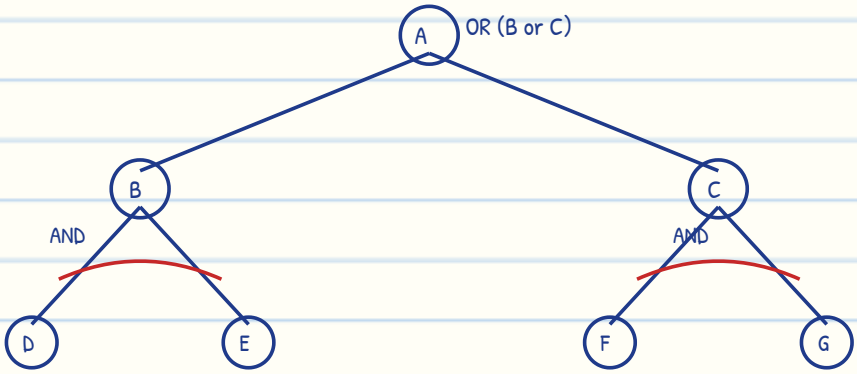


Fig 4.2 – AND-OR graph: A solved via B (needs D AND E) OR via C (needs F AND G)

➤ AO* Algorithm

1. Start with graph containing only the initial node; f = h(start).
2. Trace the ***current best partial solution graph*** (follow marked best arcs) and pick an unexpanded node n on it.
3. Expand n; for each successor compute h. Add to graph.
4. Cost of a node = min over arcs of: ***cost(arc) = Σ (c + h(child)) over AND-children*** (or single child for OR arc). Mark the best arc.
5. ***Back-propagate*** revised costs up to the start node (update all ancestors).
6. Mark node SOLVED when all children on its best arc are solved. Repeat until start is SOLVED or cost > FUTILITY.

- A* works on OR graphs only; AO* handles AND arcs (problem reduction, theorem proving, assembly-sequence planning).
- Optimal if h is admissible; avoids revisiting by cost revision.
- M.E. use: ***assembly planning*** – "assemble gearbox" = (assemble shaft sub-assembly AND assemble housing) OR (buy ready sub-unit).

> Cost eg: arc A → (B AND C) with h(B)=3, h(C)=4, each link cost 1: cost = (1+3)+(1+4) = 9. Arc A → D (OR) with h(D)=10, link 1: cost = 11. AO* selects the AND arc (9).

5. Memory-Bounded Heuristic Search

A* runs out of memory before time. Solutions:

➤ 5.1 IDA* (Iterative-Deepening A*)

- Like IDDFS but cutoff is on ***f-cost*** (g+h) instead of depth.
- Each iteration: DFS, prune nodes with f > current threshold. New threshold = smallest f that exceeded the old one.
- Complete & optimal (admissible h); memory O(bd). Drawback: repeated re-expansion when f-values are all different.

➤ 5.2 RBFS (Recursive Best-First Search)

- Recursive, linear space; keeps track of f-value of best ***alternative*** path available from any ancestor (f_limit).
- If current node's f exceeds f_limit → unwinds, replacing the node's f with best f of its children (remembers "how good this branch was").
- Optimal if h admissible; space O(bd); may re-generate nodes many times.

➤ 5.3 SMA* (Simplified Memory-Bounded A*)

- Works like A* until memory is ***full***; then drops the ***worst leaf*** (highest f) and backs its f-value up to the parent.
- Complete if solution depth fits in memory; optimal if reachable; uses all available memory.

6. Local Search Algorithms & Optimization Problems

In many problems the ***path is irrelevant***; only the final state matters (e.g. n-queens, circuit layout, job-shop scheduling, optimal design of a beam). ***Local search*** keeps a ***single current state*** and moves to neighbours.

- Advantages: very little memory (constant); can find good solutions in huge/infinite spaces.
- Useful for ***optimization problems***: find the state that maximizes/minimizes an ***objective function*** (e.g. minimize weight, maximize efficiency).

➤ State-space landscape

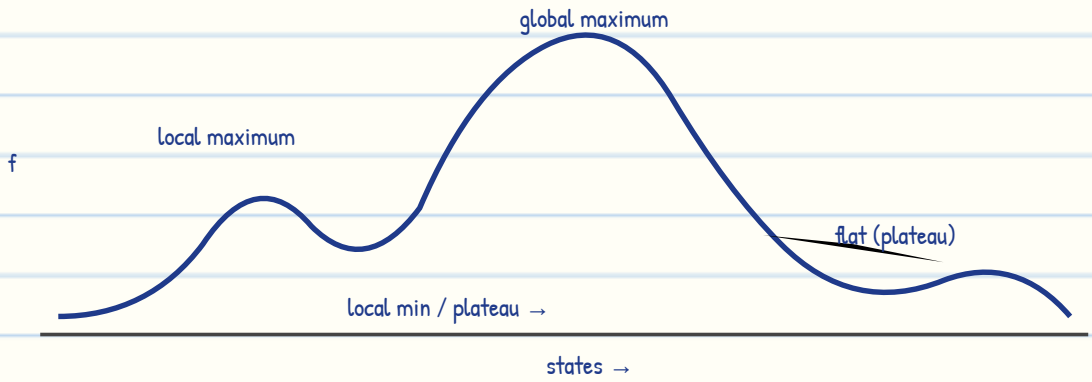


Fig 6.1 – Landscape: shoulder, local max, global max, plateau

➤ 6.1 Hill-Climbing Search

Loop that continually moves in the direction of ***increasing value*** (uphill); terminates when no neighbour has a higher value. Called ***greedy local search*** – no look-ahead beyond immediate neighbours.

```
current = initial state
loop:
  neighbour = highest-valued successor of current
  if value(neighbour) <= value(current): return current
  current = neighbour
```

➤ Problems with hill climbing

- ***Local maxima:*** peak higher than neighbours but lower than global max – search stops.
- ***Ridges:*** sequence of local maxima hard to navigate (needs diagonal moves).
- ***Plateaus / shoulders:*** flat area; no uphill direction – may wander.

➤ Variants

- ***Steepest-ascent:*** checks all neighbours, chooses the best.
- ***Stochastic:*** chooses randomly among uphill moves (probability ∝ steepness).
- ***First-choice:*** generates random successors until a better one appears – good when many neighbours.
- ***Random-restart:*** repeat hill climbing from random initial states; keeps best result – "complete" with probability → 1.

- ***Sideways moves:*** allow limited moves on plateaus.

> ***Eg. 8-queens:*** h = number of attacking pairs. Steepest-ascent solves only ≈14% of random instances, stuck in 86%; with sideways moves ≈94% solved.

➤ 6.2 Simulated Annealing

Inspired by ***annealing in metallurgy*** – heating metal then cooling slowly so atoms settle into a low-energy crystal (strong, defect-free). A very ***mechanical-engineering*** idea!

- Combines hill climbing with ***random walk*** to escape local maxima.
- Picks a ***random*** move. If it improves ($\Delta E > 0$) → always accept. If it worsens → accept with probability $e^{\Delta E/T}$.
- ***Temperature T*** is high at first (many bad moves accepted) and is lowered slowly by a ***cooling schedule***; as $T \to 0$ it behaves like hill climbing.

```
current = initial; T = schedule(0)
for t = 1,2,3...:
   T = schedule(t);  if T == 0: return current
   next = random successor of current
   ΔE = value(next) - value(current)
   if ΔE > 0: current = next
   else: current = next with probability e^(ΔE/T)
```

- If T is lowered slowly enough → finds the ***global optimum*** with probability → 1.
- Applications: VLSI layout, factory/job-shop scheduling, truss & structural optimization, travelling-salesman, tool-path planning.

Hill climbing	*Simulated annealing*
Only uphill moves	Accepts downhill moves with probability
Stuck at local max	Can escape local max
Fast, simple	Slower; needs good cooling schedule

➤ 6.3 Local Beam Search

Keeps track of ***k states*** (the beam width) instead of one.

1. Begin with k randomly generated states.

2. Generate ***all successors*** of all k states.
3. If any is a goal → stop. Else select the ***k best*** successors from the combined list and repeat.

- Differs from k parallel hill-climbs: information is ***shared*** – useful states "recruit" others ("come over here, the grass is greener").
- Problem: states can become concentrated in a small region (lack of diversity).
- ***Stochastic beam search:*** choose k successors randomly with probability proportional to value (like natural selection → leads to ***genetic algorithms***).

Algorithm	Memory	Escapes local max?	Typical use
Hill climbing	O(1)	No	Quick improvement
Simulated annealing	O(1)	Yes (probabilistic)	Layout, scheduling
Local beam	O(k)	Partly	Parallel exploration

7. Quick Revision – Important Questions & Answers

➤ Q1. Differentiate informed and uninformed search.

Uninformed: no domain knowledge, uses only problem definition (BFS, DFS). Informed: uses heuristic h(n) to guide search (Greedy, A*); usually faster & fewer nodes.

➤ Q2. Why is DFS memory efficient but BFS is not?

DFS stores only a single path with siblings → O(bm). BFS stores whole frontier → $O(b^d)$.

➤ Q3. When is A* optimal?

When h(n) is admissible (tree search) or consistent (graph search).

➤ Q4. What is an admissible heuristic? Give an example.

$h(n) \leq$ true cost to goal. e.g. straight-line distance never exceeds the road distance.

➤ Q5. State drawbacks of hill climbing and remedies.

Local maxima, ridges, plateaus. Remedies: random restart, sideways moves, stochastic variants, simulated annealing.

➤ Q6. Why does IDDFS not waste much time repeating?

In a tree with branching factor b most nodes lie at the deepest level, so upper levels re-generated are a small fraction (about 11% overhead for b=10).

➤ Q7. Difference between A* and AO*.

A* searches OR graphs for a path; AO* searches AND-OR graphs for a solution sub-graph where AND arcs require all children solved.

➤ Q8. What is the role of temperature in simulated annealing?

Controls probability $e^{\Delta E/T}$ of accepting bad moves: high T → exploration; low T → exploitation (hill climbing).

➤ Q9. Compare greedy best-first and A*.

Greedy: f=h; fast but incomplete/non-optimal. A*: f=g+h; complete and optimal with admissible h but memory heavy.

➤ Q10. Name memory-bounded heuristic searches.

IDA*, RBFS, SMA*.

➤ Formula Sheet

- BFS: time & space b^d | DFS: time b^m, space bm | DLS: b^l, bl
- IDDFS: b^d, bd | Bidirectional: $b^{d/2}$
- Greedy: f = h | A*: f = g + h | UCS: f = g
- Simulated annealing acceptance probability = $e^{\Delta E/T}$

UNIT III – Constraint Satisfaction Problems & Game Theory

AI & ML for Mechanical Engineering (23A03701)

➤ Syllabus of this unit

- Local search for constraint satisfaction problems (CSP)
- Adversarial search, games, optimal decisions & strategies in games
- The minimax search procedure, alpha-beta pruning
- Additional refinements, iterative deepening

1. Constraint Satisfaction Problems (CSP)

➤ Why CSP?

In ordinary search a state is a ***black box*** – we only know "goal or not goal". In a CSP the state has an ***internal structure*** (variables with values), so we can reject bad partial assignments early. This makes CSP solvers far faster than blind search.

➤ Definition

A CSP is defined by three components **(X, D, C)**:

- **X** = set of variables {X1, X2, ..., Xn}
- **D** = set of domains {D1, ..., Dn}; each Di is the set of allowed values for Xi
- **C** = set of constraints that specify allowed combinations of values

- ***Assignment*** – giving values to some / all variables.
- ***Consistent (legal) assignment*** – violates no constraint.
- ***Complete assignment*** – every variable has a value.
- ***Solution*** = a complete and consistent assignment.
- ***Partial assignment*** – only some variables assigned.

➤ Types of constraints

Type	Meaning	Example
Unary	restricts a single variable	SA ≠ green
Binary	relates two variables	SA ≠ WA
Global (n-ary)	involves many variables	Alldiff(X1..X9) in Sudoku
Preference (soft)	desirable, not compulsory	prefer red to green (cost)

➤ Types of variables / domains

- **Discrete finite** domain – map colouring, N-queens, scheduling.
- **Discrete infinite** – integers, strings (needs constraint language).
- **Continuous** – start/end times of jobs; solved by linear programming.

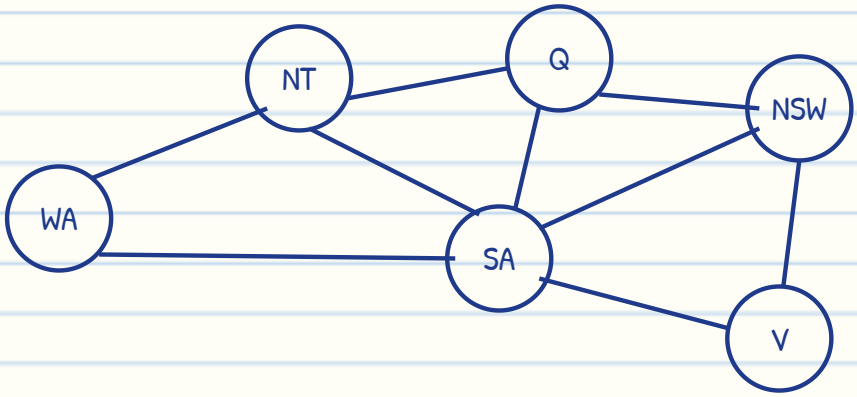
➤ Mechanical engineering applications

- Job-shop / machine scheduling (no two jobs on one machine together)
- Assembly sequencing of parts in a gearbox
- Timetabling, cutting-stock, pipe-routing, layout of components

2. Standard CSP Examples

➤ (a) Map colouring – Australia

Variables: WA, NT, Q, NSW, V, SA, T. Domain: {red, green, blue}. Constraint: adjacent regions get different colours, e.g. WA ≠ NT, WA ≠ SA, NT ≠ SA, NT ≠ Q, SA ≠ Q, SA ≠ NSW, SA ≠ V, Q ≠ NSW, NSW ≠ V.



T (Tasmania) – isolated, no edges

Constraint graph: node = variable, edge = binary constraint

> **One solution:** WA=red, NT=green, SA=blue, Q=red, NSW=green, V=red, T=any colour. T is an independent sub-problem – it can be solved separately.

➤ (b) N-Queens (4-queens)

Place N queens on N×N board so no two attack (same row, column, diagonal). **Formulation:** variable Qi = row of the queen in column i; domain {1..N}; constraints Qi ≠ Qj and |Qi – Qj| ≠ |i – j|.

➤ 4-queens solution

(Q1,Q2,Q3,Q4) = (2, 4, 1, 3) and its mirror (3, 1, 4, 2). Only 2 solutions exist for N = 4.

➤ (c) Cryptarithmetic

SEND + MORE = MONEY. Each letter a different digit. Constraints: Alldiff, column sums with carries C1..C3. Solution: S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2 (9567+1085=10652).

➤ (d) Others

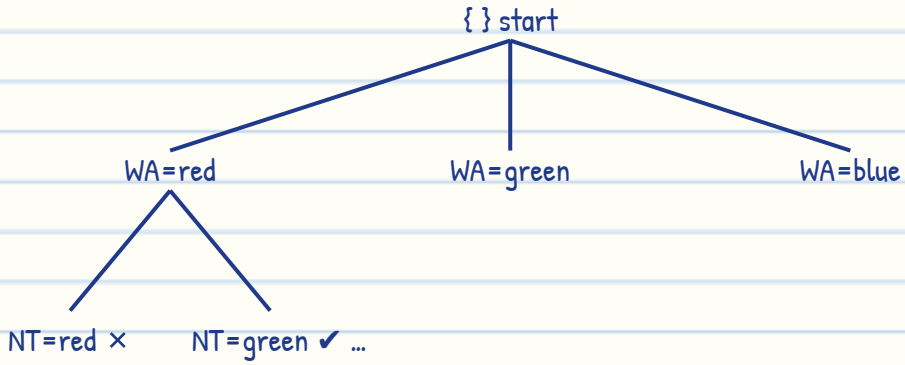
Sudoku (81 variables, Alldiff on rows, columns, boxes), Boolean satisfiability, scheduling of machining operations.

3. Solving CSPs – Backtracking Search

CSP assignments are ***commutative***: order of assignment doesn't matter ([WA=red then NT=green] same as reverse). So at each node we assign only one variable – branching factor stays d, not nd. Total leaves d^n, not $n! \cdot d^n$.

➤ Backtracking algorithm (DFS for CSP)

```
function BACKTRACK(assignment, csp):
  if assignment is complete: return assignment
  var = SELECT-UNASSIGNED-VARIABLE(csp)
  for each value in ORDER-DOMAIN-VALUES(var):
     if value consistent with assignment:
        add {var = value} to assignment
        result = BACKTRACK(assignment, csp)
        if result != failure: return result
        remove {var = value}      # undo
  return failure
```



Backtracking tree: NT=red conflicts with WA=red so we back up

Plain backtracking is still slow. Three questions improve it:

1. Which variable next? (variable ordering)
2. Which value first? (value ordering)
3. Can we detect failure early? (inference)

4. Heuristics for Backtracking

➤ Variable ordering

- **MRV – Minimum Remaining Values** (fail-first / most constrained variable): choose the variable with the fewest legal values left. E.g. after WA=red, NT=green, SA has only {blue}, so pick SA next. Exposes failure early.
- **Degree heuristic** (tie-breaker): choose the variable involved in the most constraints on unassigned variables. At the start SA (degree 5) is chosen first.

➤ Value ordering

- **LCV – Least Constraining Value**: choose the value that rules out the fewest choices for neighbouring variables, leaving maximum flexibility.

> **Remember:** for variables use "fail-first" (MRV); for values use "fail-last" (LCV) – because we need just one solution.

5. Inference – Constraint Propagation

Constraint propagation uses constraints to **reduce domains** of variables, which in turn reduces others – done as pre-processing or during search.

➤ Forward Checking (FC)

When X is assigned, delete from each unassigned neighbour Y's domain any value inconsistent with X. If any domain becomes empty → backtrack immediately.

Step	WA	NT	Q	NSW	V	SA
Initial	RGB	RGB	RGB	RGB	RGB	RGB
WA = red	R	GB	RGB	RGB	RGB	GB
Q = green	R	B	G	RB	RGB	B
V = blue	R	B	G	R	B	∅

SA's domain is empty after V=blue, so FC detects failure **before** trying more. But note NT and SA are both only {blue} and adjacent – FC doesn't notice this! We need stronger propagation.

➤ Arc Consistency (AC-3)

➤ Definition

Arc X → Y is **consistent** if for every value x in D(X) there is some value y in D(Y) satisfying the constraint between X and Y.

If a value x has no support in D(Y), delete x from D(X). Repeat; when D(X) shrinks, re-check all arcs (Z → X) pointing into X.

```
function AC-3(csp):
  queue = all arcs in csp
  while queue not empty:
    (Xi, Xj) = REMOVE-FIRST(queue)
    if REVISE(csp, Xi, Xj):
      if D(Xi) is empty: return false
      for each Xk in neighbours(Xi) - {Xj}:
        add (Xk, Xi) to queue
  return true
```

Example: X, Y ∈ {0..9}, constraint $Y = X^2$. Arc X → Y: X can only be {0,1,2,3}; arc Y → X: Y only {0,1,4,9}.
Complexity of AC-3: $O(c \cdot d^3)$ (c = no. of arcs).

- ***Node consistency*** – unary constraints satisfied.
- ***Path consistency*** – any consistent pair of 2 variables extends to a third.
- ***k-consistency*** – generalisation; strong n-consistency solves the CSP without backtracking.

MAC = Maintaining Arc Consistency: run AC-3 after every assignment during backtracking.

➤ Intelligent backtracking

- ***Chronological*** backtracking goes to the most recent variable.
- ***Backjumping*** jumps to the most recent variable in the *conflict set* – the actual cause of failure.
- ***Conflict-directed backjumping*** and ***constraint learning*** (store no-goods) refine this further.

➤ Problem structure

- Independent subproblems (Tasmania) – connected components of the constraint graph solved separately: reduces d^n to $(n/c) \cdot d^c$.
- ***Tree-structured CSP*** solved in $O(n \cdot d^2)$: pick root, order variables topologically, make arcs consistent from leaves up, assign from root down.
- ***Cutset conditioning*** – assign a cutset of variables so remaining graph becomes a tree.

6. Local Search for CSPs

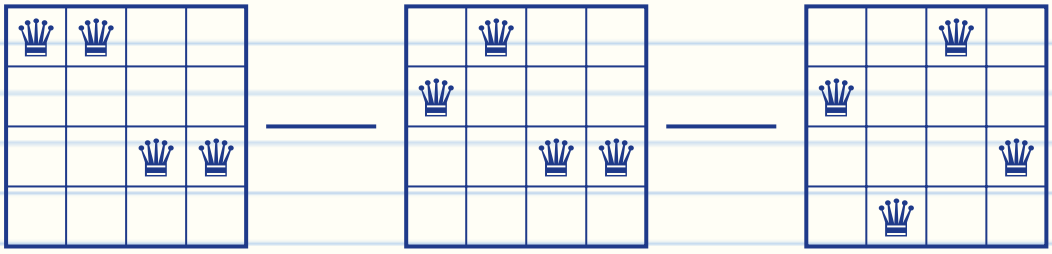
Backtracking builds a solution step by step. ***Local search*** starts with a complete assignment (usually with conflicts) and repairs it by changing one variable at a time until all constraints are satisfied. Memory needed is tiny – excellent for huge problems (e.g. million-queens solved in ~50 steps).

➤ Min-Conflicts heuristic

```
function MIN-CONFLICTS(csp, max_steps):
  current = a complete (random) assignment
  for i = 1 to max_steps:
    if current is a solution: return current
    var = randomly chosen CONFLICTED variable
    value = value of var that MINIMISES number of conflicts
    set var = value in current
  return failure
```

➤ 4-Queens by min-conflicts

Start: columns have queens in rows (1,1,3,3) → several attacks (h = conflicts). Pick a conflicted queen, move it vertically within its column to the row with fewest attacks. Typically 1–3 moves reach a solution such as (2,4,1,3).



h = 5 h = 2 h = 0 ✔

Min-conflicts reduces conflicts (h) at each step

- Landscape may have ***plateaus*** – use ***tabu search*** (remember recent moves) or ***random walk***.
- Can use ***constraint weighting***: increase weight of constraints that stay violated so search is pushed out of local minima.
- Also hill-climbing / simulated annealing (Unit II) work on CSPs with h = number of violated constraints.
- Advantage: handles ***online*** changes (re-scheduling after a machine breakdown) by repairing the current schedule.

Feature	Backtracking	Local search (min-conflicts)
State	partial assignment	complete assignment
Complete?	Yes	No (may get stuck)
Memory	depth n	O(1) – only current state
Best for	small, tight problems	large, loosely constrained

7. Adversarial Search & Games

In ***multi-agent competitive*** environments, the goals of agents conflict – e.g. chess, tic-tac-toe, checkers. The opponent's moves are unpredictable, so we need a ***strategy*** (what to do for every possible reply), not just a fixed path. Game playing was one of the first AI tasks (Shannon 1950, Turing).

Why games are hard: huge search space (chess: branching ≈ 35, depth ≈ 100 → 35^{100} nodes), and there is a time limit per move – so we must make the *best possible decision* with imperfect search.

➤ Types of games

	Deterministic	Chance (stochastic)
Perfect information	Chess, Checkers, Go, Tic-tac-toe	Backgammon, Monopoly
Imperfect information	Battleship	Bridge, Poker, Scrabble

We study ***two-player, zero-sum, deterministic, perfect-information, turn-taking*** games. Zero-sum: one player's gain = other's loss.

➤ Formal definition of a game

- ***S0*** – initial state
- ***PLAYER(s)*** – whose turn in state s (MAX or MIN)
- ***ACTIONS(s)*** – legal moves
- ***RESULT(s,a)*** – transition model
- ***TERMINAL-TEST(s)*** – is game over?
- ***UTILITY(s,p)*** – final payoff (chess: +1 win, 0 draw, −1 loss)

Two players: ***MAX*** (tries to maximise utility, moves first) and ***MIN*** (tries to minimise). The ***game tree*** has nodes = states, edges = moves; a level of one player's move is a ***ply***.

➤ Optimal decisions & strategies

In normal search the solution is a sequence of actions. In a game MAX must find a contingent strategy: a move for the initial state, then moves for every possible reply of MIN, and so on. An ***optimal strategy*** gives outcome at least as good as any other strategy *when playing against an infallible opponent.*

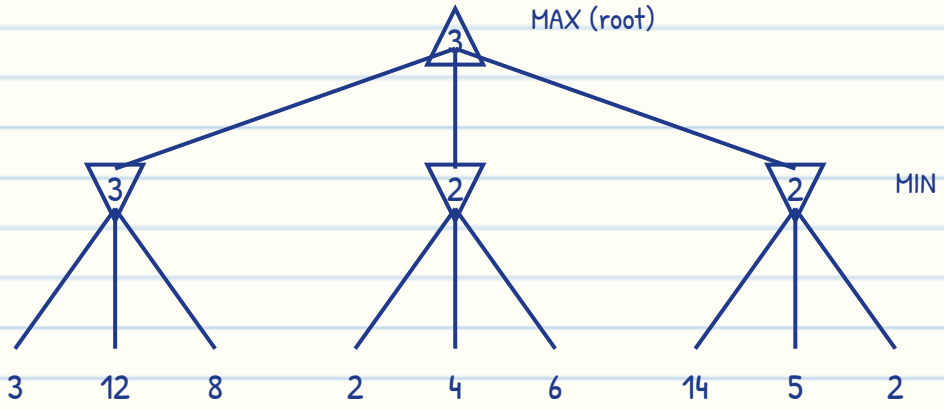
8. The Minimax Procedure

➤ Minimax value

```
MINIMAX(s) =
  UTILITY(s)                    if TERMINAL-TEST(s)
  max over a of MINIMAX(RESULT(s,a))  if PLAYER(s) = MAX
  min over a of MINIMAX(RESULT(s,a))  if PLAYER(s) = MIN
```

Idea: choose the move leading to the position with the highest ***minimax value*** – the best achievable utility assuming MIN plays optimally. Values are computed by DFS and ***backed up*** from the leaves.

```
function MINIMAX-DECISION(state):
  return argmax over a in ACTIONS(state) of MIN-VALUE(RESULT(state,a))
function MAX-VALUE(state):
  if TERMINAL-TEST(state): return UTILITY(state)
  v = -infinity
  for each a: v = MAX(v, MIN-VALUE(RESULT(state,a)))
  return v
function MIN-VALUE(state):
  if TERMINAL-TEST(state): return UTILITY(state)
  v = +infinity
  for each a: v = MIN(v, MAX-VALUE(RESULT(state,a)))
  return v
```



leaf utilities: MIN picks min of children: 3, 2, 2 → MAX picks max = 3 (move a1)

Worked minimax tree (the classic example)

Working: B = min(3,12,8) = 3; C = min(2,4,6) = 2; D = min(14,5,2) = 2. Root A = max(3,2,2) = **3**. So MAX plays a1 (towards B).

➤ Properties of minimax

Property	Value
Complete?	Yes, if the tree is finite
Optimal?	Yes, against an optimal opponent
Time	$O(b^m)$
Space	O(bm) (DFS, all actions at once) or O(m) (one at a time)

b = legal moves per position, m = max depth. For chess b≈35, m≈100 → completely infeasible, hence ***pruning***.

➤ Multi-player games

Replace single value by a ***vector*** of utilities, one per player; each player maximises own component. Alliances may form implicitly.

9. Alpha-Beta Pruning

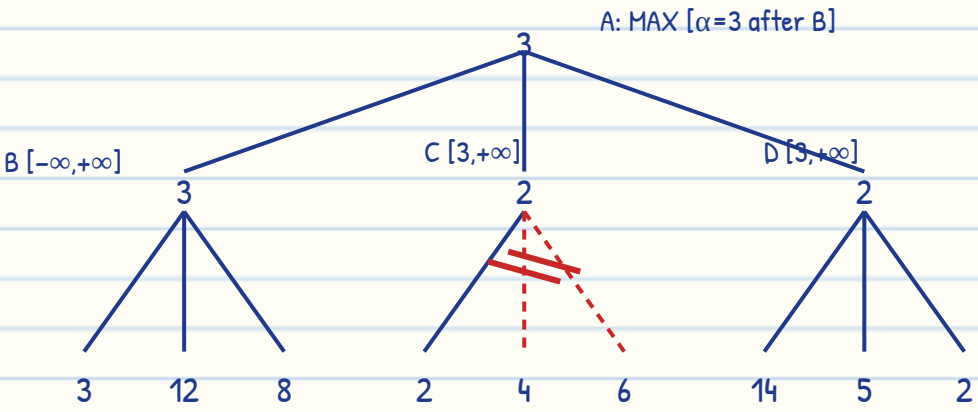
Minimax examines a number of nodes ***exponential*** in depth. Alpha-beta pruning returns the same minimax decision but ignores branches that cannot influence the final decision. It can cut the effective depth searched in half.

➤ Definitions

- α ***(alpha)*** = value of the ***best (highest)*** choice found so far for MAX along the path. Initially $-\infty$.
- β ***(beta)*** = value of the ***best (lowest)*** choice found so far for MIN along the path. Initially $+\infty$.
- ***Prune*** (stop exploring the remaining children) whenever $\alpha \geq \beta$.

```
function ALPHA-BETA-SEARCH(state):
  v = MAX-VALUE(state, -inf, +inf); return action with value v
function MAX-VALUE(state, alpha, beta):
  if TERMINAL-TEST(state): return UTILITY(state)
  v = -inf
  for each a in ACTIONS(state):
    v = MAX(v, MIN-VALUE(RESULT(state,a), alpha, beta))
    if v >= beta: return v        # beta cut-off
    alpha = MAX(alpha, v)
  return v
function MIN-VALUE(state, alpha, beta):
  if TERMINAL-TEST(state): return UTILITY(state)
  v = +inf
  for each a in ACTIONS(state):
    v = MIN(v, MAX-VALUE(RESULT(state,a), alpha, beta))
    if v <= alpha: return v       # alpha cut-off
    beta = MIN(beta, v)
  return v
```

➤ Worked example (same tree as minimax)



pruned! 4, 6 never examined

leaves of D: 14 → β=14; 5 → β=5; 2 → β=2 (all seen, no cut because 2 < 3 only at last)

Dashed red branches are pruned

Trace:

1. Visit B ($\alpha=-\infty, \beta=+\infty$): leaves 3, 12, 8 → B = 3. Back at A: α = 3.
2. Visit C with [$\alpha=3, \beta=+\infty$]: first leaf = 2 → v = 2 ≤ $\alpha(3)$ ⇒ ***cut-off***. Leaves 4 and 6 are skipped. C ≤ 2, so MAX will never choose C.
3. Visit D [3, +∞]: leaves 14 ($\beta=14$), 5 ($\beta=5$), 2 (v=2 ≤ 3 → value 2). D = 2.
4. Root = max(3, 2, 2) = 3. Same answer as minimax with 2 fewer leaf evaluations.

➤ Effectiveness & move ordering

- Pruning does not change the result – only efficiency.
- ***Worst case*** (bad ordering): no pruning, $O(b^m)$.
- ***Best case*** (perfect ordering, best moves first): $O(b^{m/2})$ – can search twice as deep in the same time. Chess: effective branching drops from 35 to about 6.
- ***Random ordering***: roughly $O(b^{3m/4})$.
- ***Move ordering heuristics:*** try captures first, then threats, forward moves, backward moves (chess); use ***killer moves*** (moves that caused cut-offs earlier at same depth); use ***iterative deepening*** results to order moves.
- ***Transposition table*** – hash table storing evaluated positions to avoid recomputing repeated states reached by different move orders.

Minimax	Alpha-beta
Explores whole tree to depth limit	Prunes irrelevant branches
$O(b^m)$	$O(b^{m/2})$ best case
Needs no α, β	Maintains α, β bounds
Same final decision	Same final decision

10. Additional Refinements

➤ (a) Imperfect real-time decisions – cutoff test

Searching to terminal states is impractical, so (Shannon 1950) we replace:

- TERMINAL-TEST → ***CUTOFF-TEST*** (e.g. depth limit d, or time up)
- UTILITY → ***EVAL*** (heuristic evaluation function estimating utility)

H-MINIMAX(s,d) = EVAL(s) if CUTOFF-TEST(s,d); else max/min of H-MINIMAX over successors with d+1.

➤ (b) Evaluation functions

Should (1) agree with true utility on terminal states, (2) be fast, (3) correlate with the real chance of winning.

Typically a ***weighted linear function***:

$EVAL(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$

Chess: f = (no. of white pieces − black pieces) of each type; weights pawn=1, knight/bishop=3, rook=5, queen=9; extra features: king safety, pawn structure, mobility. Weights can be ***learned*** from experience.

Tic-tac-toe example: EVAL = (rows/cols/diagonals still open for MAX) − (those open for MIN).

➤ (c) Quiescence search

EVAL is reliable only in ***quiescent*** positions – ones unlikely to change wildly soon (e.g. no pending capture). At the cutoff depth, if the position is non-quiescent, extend the search until it becomes quiet.

➤ (d) Horizon effect

A damaging move by the opponent is unavoidable, but the program delays it with delaying tactics (useless checks, sacrifices) so that it falls ***beyond the search horizon*** – and wrongly believes the problem is gone. Remedies: ***singular extensions*** (extend search for a clearly-best move), quiescence search, deeper search.

➤ (e) Forward pruning

- ***Beam search*** – consider only the n best moves at each ply (risky: best move may be pruned).
- ***ProbCut*** – prune moves statistically likely to be outside the (α, β) window.
- Late-move reductions: search later moves in the list to lesser depth.

➤ (f) Table look-up

Use ***opening books*** (first few moves from human experience) and ***endgame databases*** (perfect play computed backwards for few pieces left).

➤ (g) Stochastic games (chance)

Games with dice (backgammon): add ***chance nodes*** between MAX and MIN levels. Use ***expectiminimax***:

```
EXPECTIMINIMAX(s) =
  UTILITY(s)                          if terminal
  max over a  EXPECTIMINIMAX(RESULT(s,a)) if MAX
  min over a  EXPECTIMINIMAX(RESULT(s,a)) if MIN
  sum over r  P(r) · EXPECTIMINIMAX(RESULT(s,r)) if CHANCE
```

Coin-flip chance node (P=0.5 each) over MIN values 2 and 4: expected value = 0.5×2 + 0.5×4 = **3**. Complexity $O(b^m n^m)$, n = number of dice rolls – deep look-ahead is hard. Evaluation functions must be positive linear transforms of win probability. Monte-Carlo simulation (rollouts) is an alternative; Monte-Carlo tree search (MCTS) drives AlphaGo.

➤ (h) Partially observable games

Games like Bridge, Poker involve hidden information – handled by averaging over possible hidden states (card-deal sampling) or belief-state reasoning.

11. Iterative Deepening in Games

Because the time per move is fixed (e.g. 2 minutes), we do not know in advance what depth will finish. ***Iterative deepening search (IDS)***: run depth-limited alpha-beta with depth 1, then 2, then 3 ... until time runs out, and return the move from the deepest completed iteration.

```
function ITERATIVE-DEEPENING(state, time_limit):
  best = none
  for depth = 1, 2, 3, ...:
    if time is up: break
    best = ALPHA-BETA-SEARCH(state, depth)
  return best
```

Advantage	Explanation
Anytime algorithm	always has a move ready when the clock expires
Little overhead	repeated shallow levels cost small fraction because most nodes are at the bottom level (≈ b/(b−1) times extra)
Better move ordering	best move of depth d−1 is tried first at depth d → more alpha-beta cut-offs
Memory-efficient	DFS-like space O(bd)

Combined with ***transposition tables*** (store best move per position), results of earlier iterations are reused.

12. Quick Summary

- CSP = (X, D, C); solution is complete + consistent assignment.
- Backtracking + MRV + degree + LCV + forward checking / AC-3 (MAC) solves CSPs efficiently.
- Local search (min-conflicts) repairs a complete assignment – great for large CSPs.
- Minimax gives optimal play against an optimal opponent; alpha-beta prunes (prune when $\alpha \geq \beta$), best case $O(b^{m/2})$.
- Real games need cutoff + EVAL, quiescence, horizon-effect handling, chance nodes (expectiminimax), iterative deepening.

13. Important Exam Questions & Short Answers

Q1. Define CSP. Give an example.

A problem with variables X, domains D and constraints C; solution is a complete consistent assignment. Example: map colouring of Australia with 3 colours, adjacent regions different.

Q2. Differentiate forward checking and arc consistency.

FC only checks neighbours of the *just-assigned* variable and deletes inconsistent values; AC-3 propagates repeatedly through all arcs, so it detects failures FC misses (e.g. adjacent NT, SA both {blue}).

Q3. What are the MRV, degree and LCV heuristics?

MRV: variable with fewest legal values. Degree: variable with most constraints on unassigned variables (tie-break). LCV: value that rules out fewest values in neighbours.

Q4. Why is min-conflicts effective?

It repairs a complete assignment by moving a conflicted variable to its least-conflict value; the solution density is high in many problems, so it needs few steps and O(1) extra memory. Fails on plateaus unless tabu / random walk is used.

Q5. What is a zero-sum game? What are MAX and MIN?

A game where total payoff is constant (one's gain is other's loss). MAX maximises, MIN minimises the utility of the terminal state.

Q6. Write minimax and its complexity.

Recursive DFS; value of a MAX node = max of children, MIN node = min of children. Time $O(b^m)$, space $O(bm)$.

Q7. Explain alpha-beta pruning with an example.

Maintain α (best for MAX) and β (best for MIN); stop expanding a node when $\alpha \geq \beta$. In the example tree, leaves 4 and 6 under C are pruned after seeing 2 because $2 \leq \alpha = 3$. Gives same result as minimax; best case $O(b^{m/2})$.

Q8. What is the horizon effect and how is it reduced?

Delaying an inevitable loss beyond the search depth so it looks avoided. Reduced by quiescence search and singular extensions.

Q9. Why iterative deepening in game playing?

Time limited; it gives an anytime answer, small overhead, and improves move ordering so alpha-beta prunes more.

Q10. What is expectiminimax?

Extension of minimax for stochastic games with chance nodes whose value is the probability-weighted average of children.

~ End of Unit III ~ All the best! ~

UNIT IV – Knowledge & Reasoning

AI & ML for Mechanical Engineering (23A03701)

➤ Syllabus Map

- Statistical reasoning: probability, Bayes' theorem, certainty factors, Bayesian networks, Dempster-Shafer, fuzzy logic
- Knowledge representation: rule-based, procedural vs declarative, logic programming, forward & backward reasoning

1. Why Statistical Reasoning?

Real-world engineering data is ***uncertain, noisy and incomplete***. Sensors drift, a bearing may or may not be worn, a crack may or may not be present. Pure logic (TRUE/FALSE) cannot handle this, so AI uses reasoning under uncertainty.

➤ Sources of uncertainty

- Imprecise / noisy sensor readings (thermocouple error)
- Incomplete knowledge (not all symptoms observed)
- Vague concepts ("high temperature", "slightly worn")
- Unreliable expert rules ("usually", "often")

➤ Tools we study

Tool	Handles
Probability & Bayes	Randomness, belief update with evidence
Certainty Factors	Expert's confidence in a rule (MYCIN)
Bayesian Network	Many variables with dependencies
Dempster-Shafer	Ignorance (not knowing) vs disbelief
Fuzzy Logic	Vagueness / degrees of truth

2. Probability Basics

➤ Definition

Probability of event A: **$P(A) = (\text{favourable outcomes}) / (\text{total outcomes})$**, with $0 \leq P(A) \leq 1$, P(sure event) = 1, P(impossible) = 0.

➤ Important rules

- Complement: **$P(\neg A) = 1 - P(A)$**
- Addition: **$P(A \cup B) = P(A) + P(B) - P(A \cap B)$**
- Mutually exclusive: $P(A \cap B) = 0$
- Conditional: **$P(A \mid B) = P(A \cap B) / P(B)$**, $P(B) > 0$
- Product rule: $P(A \cap B) = P(A|B) \cdot P(B)$
- Independent events: $P(A \cap B) = P(A) \cdot P(B)$
- Total probability: $P(B) = \Sigma P(B|A_i) \cdot P(A_i)$

> ***Example.*** A lot has 100 shafts: 8 are defective. Probability that a randomly drawn shaft is good = 1 – 8/100 = 0.92. Two shafts drawn with replacement, both defective = 0.08 × 0.08 = 0.0064.

➤ Joint & marginal probability

Joint P(A,B) = both occur together. Marginal P(A) = sum of joint probabilities over all values of B. A full ***joint probability table*** answers any query but grows as 2^n for n Boolean variables – that is why Bayesian networks are needed.

3. Bayes' Theorem

➤ Bayes' Rule

$$P(H \mid E) = P(E \mid H) \cdot P(H) / P(E)$$

- P(H) = ***prior*** – belief in hypothesis before evidence
- P(E|H) = ***likelihood*** – chance of evidence if H is true
- P(H|E) = ***posterior*** – updated belief after evidence
- P(E) = P(E|H)P(H) + P(E|¬H)P(¬H) (normaliser)

Derivation: $P(H \cap E) = P(H|E)P(E) = P(E|H)P(H)$. Equate and divide by $P(E)$.

➤ Worked Example 1 – Crack detection

1% of turbine blades have a crack. An ultrasonic test detects a crack with 95% probability if present (sensitivity), and gives a false alarm 4% of the time on healthy blades. A blade tests ***positive***. P(crack | positive)?

$P(H)=0.01$, $P(E|H)=0.95$, $P(E|\neg H)=0.04$

$P(E) = 0.95\times0.01 + 0.04\times0.99 = 0.0095 + 0.0396 = 0.0491$

$P(H|E) = 0.0095 / 0.0491 = 0.1935 \approx 19.4\ \%$

Even a positive test gives only ~19% – because cracks are rare (low prior).

➤ Worked Example 2 – Machine source

Machine M1 makes 60% of parts (3% defective), M2 makes 40% (5% defective). A defective part is found. Probability it came from M1?

$P(D) = 0.6\times0.03 + 0.4\times0.05 = 0.018 + 0.020 = 0.038$

$P(M1|D) = 0.018/0.038 = 0.4737$; so $P(M2|D) = 0.5263$.

➤ Limitations of pure Bayes

- Needs many prior and conditional probabilities (hard to obtain)
- Assumes independence of evidences (naive Bayes) otherwise table explodes
- Cannot represent ignorance – all probability must be assigned

4. Certainty Factors & Rule-Based Systems

Developed for the medical expert system ***MYCIN*** (diagnosing blood infections). Experts give confidence numbers instead of exact probabilities.

➤ Certainty Factor (CF)

$$CF[h, e] = MB[h, e] - MD[h, e]$$

- MB = measure of belief (0 to 1) – evidence supports h
- MD = measure of disbelief (0 to 1) – evidence opposes h
- CF ranges from **–1** (absolutely false) through **0** (no information) to **+1** (absolutely true)

➤ Rule form

IF engine_vibration is high AND bearing_temp is high
THEN bearing_failure CF = 0.7

➤ Combining rules

- ***Conjunction*** (AND): CF(premise) = $\min(CF_1, CF_2)$
- ***Disjunction*** (OR): CF(premise) = $\max(CF_1, CF_2)$
- ***Rule propagation:*** CF(conclusion) = CF(rule) × CF(premise)
- ***Two rules, same conclusion*** (CF_1, CF_2):
  - both positive: $CF = CF_1 + CF_2(1 - CF_1)$
  - both negative: $CF = CF_1 + CF_2(1 + CF_1)$
  - opposite signs: $CF = (CF_1 + CF_2) / (1 - \min(|CF_1|,|CF_2|))$

➤ Example

Rule 1: IF vibration high (CF 0.8) AND temp high (CF 0.6) THEN failure, rule CF = 0.7.

Premise = min(0.8, 0.6) = 0.6 → CF_1 = 0.7 × 0.6 = 0.42

Rule 2: IF oil_debris present (CF 0.9) THEN failure, rule CF = 0.5 → CF_2 = 0.45

Combined = 0.42 + 0.45(1 − 0.42) = 0.42 + 0.261 = 0.681

➤ Rule-based system architecture

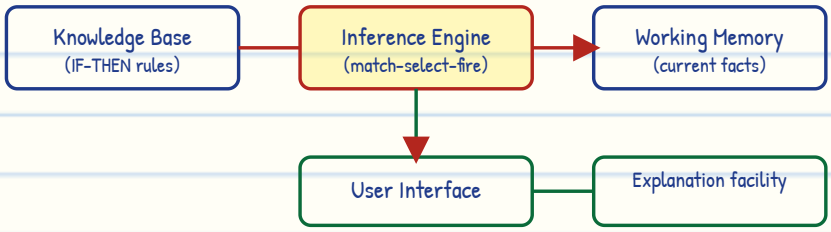


Fig 1: Components of a rule-based (expert) system

➤ Advantages / disadvantages

- \+ Easy to understand, modular, explains its reasoning (why / how)
- \+ Handles uncertainty approximately
- − CF are ad-hoc (not rigorous probability); rule interactions can mislead
- − Large rule bases hard to maintain

5. Bayesian Networks (Belief Networks)

➤ Definition

A ***Bayesian network*** is a directed acyclic graph (DAG) in which nodes are random variables and an arrow $X \rightarrow Y$ means X directly influences Y. Each node has a ***conditional probability table (CPT)*** $P(X_i \mid Parents(X_i))$.

➤ Joint distribution

$$P(X_1,..,X_n) = \Pi\ P(X_i \mid Parents(X_i))$$

Needs far fewer numbers than the full 2^n joint table.

➤ Example: machine overheating

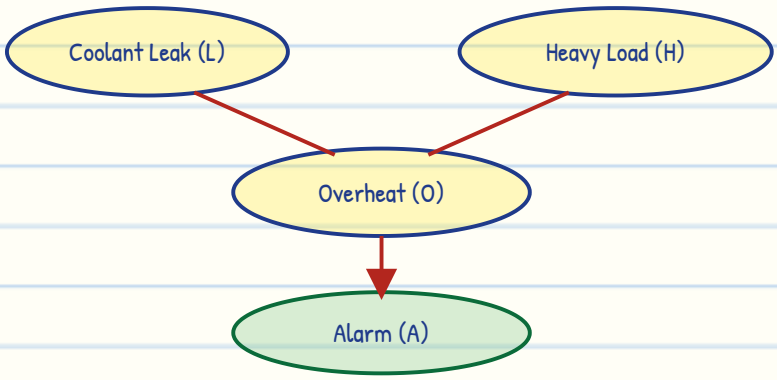


Fig 2: Bayesian network – L and H cause O; O triggers A

Node	CPT values
P(L)	0.02
P(H)	0.30
P(O | L,H)	L,H: 0.95 | L,¬H: 0.80 | ¬L,H: 0.30 | ¬L,¬H: 0.01
P(A | O)	O: 0.90 | ¬O: 0.05

➤ Compute P(L, H, O, A)

P(L=T, H=T, O=T, A=T) = P(L)·P(H)·P(O|L,H)·P(A|O)

= 0.02 × 0.30 × 0.95 × 0.90 = 0.00513

➤ Compute P(O) (marginal)

P(L,H)=0.006, P(L,¬H)=0.014, P(¬L,H)=0.294, P(¬L,¬H)=0.686

P(O) = 0.006(0.95)+0.014(0.80)+0.294(0.30)+0.686(0.01)

= 0.0057+0.0112+0.0882+0.00686 = 0.11196

P(A) = 0.90(0.11196) + 0.05(0.88804) = 0.10076 + 0.04440 = **0.1452**

➤ Inference types

- ***Causal*** (top-down): cause → effect, e.g. P(A | L)
- ***Diagnostic*** (bottom-up): effect → cause, e.g. P(L | A) using Bayes
- ***Explaining away:*** if overheating is explained by heavy load, belief in leak drops

Methods: exact (enumeration, variable elimination), approximate (sampling). Applications: fault diagnosis, reliability, condition monitoring, medical diagnosis.

6. Dempster-Shafer Theory (DST)

A generalisation of probability that allows ***"I don't know"***. Evidence is assigned not to single outcomes but to ***subsets of hypotheses***. Also called the theory of belief functions / evidence theory.

➤ Key terms

- ***Frame of discernment*** **Θ**: set of all mutually exclusive possible hypotheses, e.g. Θ = {Bearing, Gear, Shaft}
- ***Power set 2^Θ***: all subsets of Θ (2^3 = 8 here)
- ***Basic probability assignment (mass) m***: m: $2^{\Theta} \rightarrow [0,1]$, $m(\varnothing)=0$, $\Sigma\, m(A) = 1$
- ***Belief Bel(A)*** = $\Sigma\, m(B)$ for all $B \subseteq A$ (total support, lower bound)
- ***Plausibility Pl(A)*** = $\Sigma\, m(B)$ for all $B \cap A \neq \varnothing$ = 1 – Bel(¬A) (upper bound)
- ***Belief interval*** [Bel(A), Pl(A)]; width = ignorance

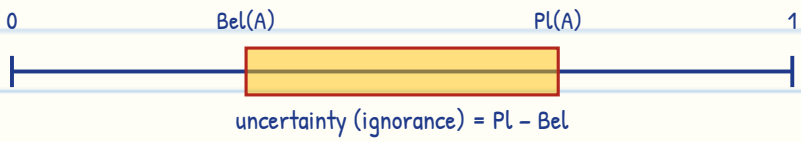


Fig 3: Belief interval

➤ Example – single source

Sensor 1 says fault is in Bearing or Gear: $m_1(\{B,G\}) = 0.6$, $m_1(\Theta) = 0.4$ (the rest is ignorance). Bel({B,G}) = 0.6; Pl({B,G}) = 0.6 + 0.4 = 1.0. Bel({B}) = 0; Pl({B}) = 1.0.

➤ Dempster's rule of combination

$$m_{12}(C) = \Sigma_{A \cap B = C}\ m_1(A) \cdot m_2(B) \ / \ (1 - K)$$

where conflict $K = \Sigma_{A \cap B = \varnothing}\ m_1(A) \cdot m_2(B)$. Dividing by (1–K) re-normalises.

➤ Worked example

Θ = {B, G, S}. Sensor 1: $m_1(\{B,G\}) = 0.6$, $m_1(\Theta) = 0.4$. Sensor 2: $m_2(\{G,S\}) = 0.7$, $m_2(\Theta) = 0.3$.

	$m_2\{G,S\}=0.7$	$m_2\Theta=0.3$
$m_1\{B,G\}=0.6$	{G} : 0.42	{B,G} : 0.18
$m_1\Theta=0.4$	{G,S} : 0.28	Θ : 0.12

No empty intersections → K = 0, so no normalising.

Result: $m(\{G\})=0.42$, $m(\{B,G\})=0.18$, $m(\{G,S\})=0.28$, $m(\Theta)=0.12$

Bel({G}) = 0.42, Pl({G}) = 1.0 (all sets contain G). Bel({B,G}) = 0.42+0.18 = 0.60; Pl({B,G}) = 1.0.

➤ DST vs Bayesian probability

Bayesian	Dempster-Shafer
Probability on single outcomes	Mass on subsets of outcomes
P(A)+P(¬A)=1 always	Bel(A)+Bel(¬A) ≤ 1
Ignorance spread equally (prior)	Ignorance kept explicit in m(Θ)
Needs priors	No priors needed
Single number	Interval [Bel, Pl]

Use: sensor fusion, multi-sensor fault diagnosis, decision under ignorance.

7. Fuzzy Logic

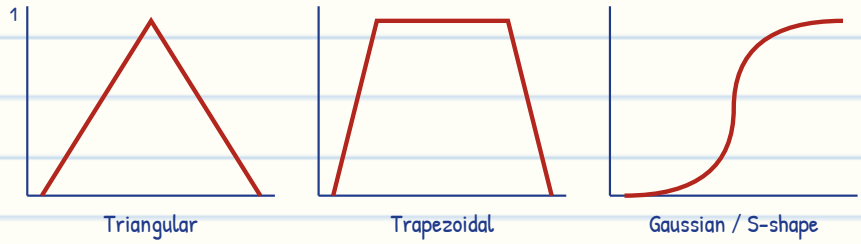
Proposed by **Lotfi Zadeh (1965)**. Classical (crisp) logic says an element is either in a set or not. Fuzzy logic allows degrees of membership between 0 and 1, modelling human-like vague terms: "warm", "fast", "high pressure".

➤ Fuzzy set

A fuzzy set A in universe X is a set of pairs: $A = \{ (x, \mu_A(x)) \mid x \in X \}$, where $\mu_A(x) \in [0,1]$ is the membership function.

Crisp set: μ is only 0 or 1. Example: "Hot" = {(30°C,0.2), (40°C,0.6), (50°C,1.0)}.

➤ Common membership functions



Triangular: μ(x)=0 for x≤a, (x−a)/(b−a) for a≤x≤b, (c−x)/(c−b) for b≤x≤c, 0 for x≥c.

➤ Fuzzy set operations (Zadeh)

Operation	Formula
Union (OR)	μ_{A∪B}(x) = max[μ_A(x), μ_B(x)]
Intersection (AND)	μ_{A∩B}(x) = min[μ_A(x), μ_B(x)]
Complement (NOT)	μ_{A'}(x) = 1 − μ_A(x)
Equality / subset	A=B if μ_A=μ_B ∀x; A⊆B if μ_A ≤ μ_B

Example. A = {0.2/a, 0.7/b, 1/c}, B = {0.5/a, 0.4/b, 0.9/c}

A∪B = {0.5/a, 0.7/b, 1/c}; A∩B = {0.2/a, 0.4/b, 0.9/c}; $A^{'}$ = {0.8/a, 0.3/b, 0/c}

➤ Crisp vs fuzzy

- Crisp: temperature 40°C is "hot" TRUE/FALSE with a sharp boundary
- Fuzzy: 40°C is hot to 0.6 degree and warm to 0.4 degree – smooth transition

8. Fuzzy Inference System (FIS)

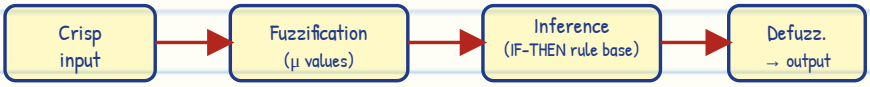


Fig 4: Fuzzy logic controller (Mamdani)

➤ Steps

- **1. Fuzzification:** convert crisp inputs to membership degrees of linguistic variables (Cold, Warm, Hot)

- **2. Rule evaluation:** apply IF-THEN rules; AND = min, OR = max
- **3. Aggregation:** combine the clipped output sets of all rules (max)
- **4. Defuzzification:** convert the fuzzy output to one crisp value

➤ Defuzzification methods

- **Centroid (centre of gravity):** $z^* = \int \mu(z) \cdot z \, dz / \int \mu(z) \, dz$ - most used
- Weighted average (discrete): $z^* = \Sigma \mu_i \cdot z_i / \Sigma \mu_i$
- Mean of maxima, smallest / largest of maxima, bisector

➤ Example – Fan speed controller for a motor

Input: temperature T. Sets: Cold, Warm, Hot. Output: fan speed Slow (20%), Medium (50%), Fast (90%).

R1: IF T is Cold THEN speed is Slow
R2: IF T is Warm THEN speed is Medium
R3: IF T is Hot THEN speed is Fast

At T = 65°C fuzzification gives: Cold = 0, Warm = 0.7, Hot = 0.3.

Weighted average: speed = (0.7×50 + 0.3×90) / (0.7+0.3) = (35+27)/1 = 62 %

➤ Rule with two inputs

IF temperature is High AND pressure is Medium THEN valve opening is Large. If μ_High(T)=0.8, μ_Med(P)=0.5 → firing strength = min(0.8,0.5) = 0.5, so the Large set is clipped at 0.5.

➤ Applications in Mechanical Engg.

- Washing machine, air conditioner, refrigerator controllers
- Automatic gearbox / ABS brakes, cruise control
- Robot path planning, CNC process parameter selection
- Furnace temperature control, fault diagnosis

➤ Pros & cons

- + No exact mathematical model needed; easy to tune by expert knowledge
- + Robust to noisy inputs
- – Rule base grows with inputs; membership design is subjective; no learning ability (hence neuro-fuzzy)

9. AI for Knowledge Representation (KR)

➤ Knowledge Representation

KR is the way of ***encoding knowledge about the world in a form a computer can store and reason with*** to solve complex tasks.

➤ Kinds of knowledge

- ***Objects*** (facts): a bolt is a fastener
- ***Events***: pump failed at 10:00
- ***Performance***: how to do things
- ***Meta-knowledge***: knowledge about knowledge

➤ Requirements of a good KR scheme

- ***Representational adequacy*** – can represent all needed knowledge
- ***Inferential adequacy*** – can derive new knowledge
- ***Inferential efficiency*** – can direct inference to promising paths
- ***Acquisitional efficiency*** – easy to add new knowledge

➤ Techniques of KR

Technique	Idea	Example
Logical (propositional / predicate)	Facts as logical sentences	∀x Motor(x) → HasRotor(x)
Semantic network	Graph of nodes and labelled links	Car –is-a→ Vehicle
Frames	Slot-filler structures with inheritance	Frame Pump: type, flow, head
Production rules	IF-THEN rules	IF temp>90 THEN alarm

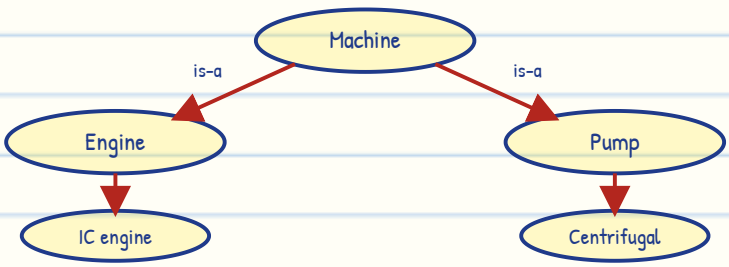


Fig 5: Semantic network (inheritance)

10. Rule-Based Knowledge Representation

Knowledge is stored as ***production rules: IF (condition) THEN (action/conclusion)***. The condition part is the *antecedent* (LHS) and the conclusion is the *consequent* (RHS).

```
R1: IF  noise = abnormal AND temp = high  THEN bearing = faulty
R2: IF  bearing = faulty                  THEN action = replace_bearing
R3: IF  oil_level = low                   THEN action = add_oil
```

➤ Components

- ***Rule base*** (long-term memory), ***working memory*** (facts about the current case), ***inference engine*** (cycle: match → conflict resolution → act)

➤ Conflict-resolution strategies

- Specificity (more specific rule first), recency, priority, first-match

➤ Pros and cons

- \+ Modular, natural, easy to update, transparent
- – Slow with many rules, hard to see rule interactions, poor at learning

11. Procedural vs Declarative Knowledge

Procedural ("know-how")	Declarative ("know-what")
Describes **how** to do something	Describes **what** is true (facts)
Control information embedded in knowledge	Control separate from knowledge
Written as procedures / programs / code	Written as facts, rules, logic statements
Efficient to run, hard to modify	Easy to modify, less efficient
Hard to reuse for other purposes	Same fact can answer many queries
Eg: algorithm to start an engine	Eg: "Engine has 4 cylinders"
Languages: C, Java, Python	Languages: Prolog, rule engines

Real systems mix both: facts are declarative, inference/search strategy is procedural.

12. Logic Programming

A programming style where the program is a ***set of logical statements (facts and rules)*** and computation is logical inference (theorem proving). The programmer states *what* is true; the system decides *how* to answer queries. Best known language: ***Prolog*** (PROgramming in LOGic), based on Horn clauses and resolution with unification and backtracking.

➤ Prolog building blocks

- ***Fact:*** unconditional truth - `motor(m1).`
- ***Rule:*** head :- body. (head is true if body is true) - `:-` means "if", comma means AND, semicolon means OR
- ***Query:*** a question - `?- faulty(m1).`
- Variables begin with a capital letter (X, Machine); constants with lower case
- Every clause ends with a full stop

➤ Example program

```
% facts
vibration_high(m1).
temp_high(m1).
temp_high(m2).
oil_low(m3).

% rules
bearing_faulty(X) :- vibration_high(X), temp_high(X).
needs_service(X) :- bearing_faulty(X).
needs_service(X) :- oil_low(X).
```

Query `?- needs_service(m1).` → ***true*** (via bearing_faulty).

Query `?- needs_service(Z).` → Z = m1 ; Z = m3. (backtracking gives all answers)

Query `?- bearing_faulty(m2).` → ***false*** (no vibration fact; closed-world assumption)

➤ Key concepts

- ***Unification:*** matching terms by binding variables, e.g. needs_service(X) with needs_service(m1) gives X = m1

- **Resolution:** inference rule used to derive new clauses / prove goals
- **Backtracking:** if a path fails, return to the last choice point and try the next clause
- **Horn clause:** clause with at most one positive literal – $A \leftarrow B_1 \wedge \ldots \wedge B_n$
- **Closed-world assumption:** anything not provable is taken as false (negation as failure)

➤ Recursion example

```
parent(a,b).  parent(b,c).
ancestor(X,Y) :- parent(X,Y).
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
```

Applications: expert systems, natural-language processing, planning, theorem proving, database queries.

13. Forward and Backward Reasoning

➤ Forward chaining

Data-driven: start from known facts, fire rules whose conditions match, add conclusions, repeat until goal reached or nothing new.

➤ Backward chaining

Goal-driven: start from the goal (hypothesis), find rules that conclude it, make their conditions new sub-goals, until sub-goals match facts.

➤ Rule base

```
R1: IF A AND B THEN C
R2: IF C THEN D
R3: IF D AND E THEN F
Facts: A, B, E.   Goal: F
```

Forward: facts {A,B,E} → R1 fires, adds C → R2 fires, adds D → R3 fires (D,E known), adds F. Goal reached.

Backward: Goal F ← R3 needs D and E. E is a fact ✔. D ← R2 needs C. C ← R1 needs A, B – both facts ✔. So F proved.

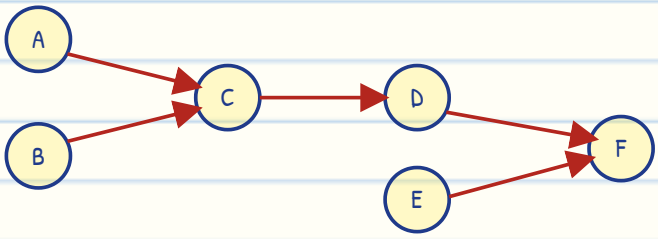


Fig 6: Inference graph for the rule base

➤ Comparison

Forward chaining	Backward chaining
Data-driven, bottom-up	Goal-driven, top-down
Starts from facts → conclusions	Starts from goal → supporting facts
Good for monitoring, planning, design, control	Good for diagnosis, proving a specific hypothesis
May derive many irrelevant facts	Explores only relevant rules
Eg: OPS5, CLIPS, process control	Eg: Prolog, MYCIN
Breadth-first nature	Depth-first nature

Bidirectional / mixed: combine both – work from facts and goal until they meet.

14. Quick Revision Formulas

- Bayes: $P(H|E) = P(E|H)P(H)/P(E)$
- CF: CF = MB – MD; premise AND = min, OR = max; conclusion = CF_rule × CF_premise
- Combine CFs (both +ve): $CF_1 + CF_2(1 - CF_1)$
- BN: $P(x_1..x_n) = \Pi P(x_i|\text{parents})$
- DST: $Bel(A)=\Sigma_{B\subseteq A}m(B)$; $Pl(A)=\Sigma_{B\cap A\neq\emptyset}m(B)$; K = conflict mass
- Fuzzy: U = max, ∩ = min, complement = 1 – μ; centroid $z^* = \Sigma\mu z/\Sigma\mu$

15. Important Exam Questions (Short Q & A)

Q1. State Bayes' theorem and explain each term.

$P(H|E)=P(E|H)P(H)/P(E)$: posterior = likelihood × prior / evidence probability. It updates belief in H after observing E.

Q2. What is a certainty factor? What is its range?

A number expressing the confidence in a hypothesis given evidence, CF = MB − MD, ranging from −1 to +1.

Q3. What is a Bayesian network? Why is it useful?

A DAG of variables with CPTs; it gives a compact representation of the joint distribution using conditional independence, avoiding a 2^n table.

Q4. Differentiate belief and plausibility in Dempster-Shafer theory.

Belief = total evidence that directly supports A (lower bound). Plausibility = evidence that does not contradict A (upper bound). Pl − Bel = ignorance.

Q5. What is the difference between fuzzy and crisp sets?

Crisp: membership is 0 or 1 with sharp boundary. Fuzzy: membership is any value in [0,1], giving gradual transition.

Q6. List the stages of a fuzzy inference system.

Fuzzification → rule evaluation (inference) → aggregation → defuzzification.

Q7. Distinguish procedural and declarative knowledge.

Procedural = how-to, embedded control (code). Declarative = facts, separate from control (Prolog, rules).

Q8. Compare forward and backward chaining.

Forward is data-driven, bottom-up; backward is goal-driven, top-down. Diagnosis uses backward; monitoring uses forward.

Q9. What is Prolog? Write a fact and a rule.

A logic programming language. Fact: `motor(m1).` Rule: `hot(X) :- motor(X), temp_high(X).`

Q10. Write about the characteristics of a good knowledge representation scheme.

Representational adequacy, inferential adequacy, inferential efficiency and acquisitional efficiency.

Long questions to prepare:

- Explain Bayes' theorem with a numerical example and its use in fault diagnosis.
- Explain Dempster-Shafer theory and Dempster's rule of combination with an example.
- Explain the architecture of a fuzzy logic controller with a mechanical example.
- Explain forward and backward reasoning with an example; compare them.
- Describe rule-based systems and certainty-factor propagation.

~ End of Unit IV ~ All the best! ~

UNIT V – Introduction to Machine Learning

AI & ML for Mechanical Engineering (23A03701)

➤ Syllabus Checklist

ML as a sub-discipline of AI · Supervised · Unsupervised · Reinforcement learning · Classification · Regression · Clustering · Neural networks & deep learning

1. What is Machine Learning?

➤ Definition (Tom Mitchell, 1997)

A program ***learns*** from experience ***E*** with respect to task ***T*** and performance measure ***P***, if its performance on T, measured by P, improves with experience E.

- ***Arthur Samuel (1959):*** ML gives computers the ability to learn without being explicitly programmed.
- Traditional programming: ***Data + Rules → Answers***
- Machine learning: ***Data + Answers → Rules (model)***

Example: Spam filter. T = classify mail as spam / not spam, E = mails labelled by users, P = % of mails classified correctly.

Mechanical example: T = predict bearing failure, E = past vibration records with failure dates, P = prediction accuracy.

➤ AI vs ML vs Deep Learning

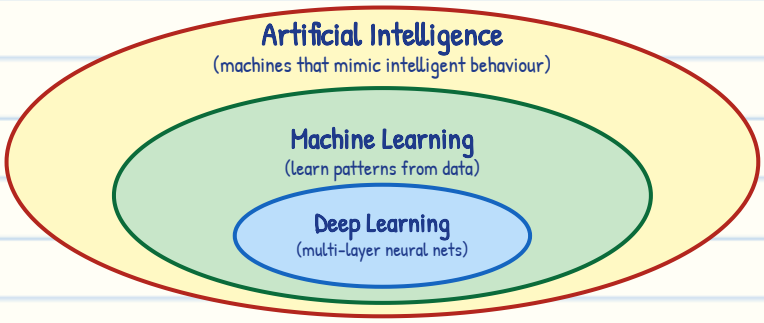
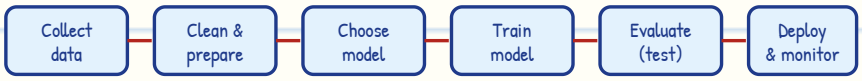


Fig 1: DL ⊂ ML ⊂ AI

➤ Why ML for Mechanical Engineers?

- Large sensor data (vibration, temperature, pressure) from machines, CNC, IoT.
- Physics-based models are slow / unknown → data-driven models are fast.
- Applications: predictive maintenance, fault diagnosis, tool-wear prediction, CFD/FEA surrogate models, material property prediction, robotics & control, additive-manufacturing quality, design optimisation.

➤ Steps in a Machine Learning Project



2. Types of Machine Learning

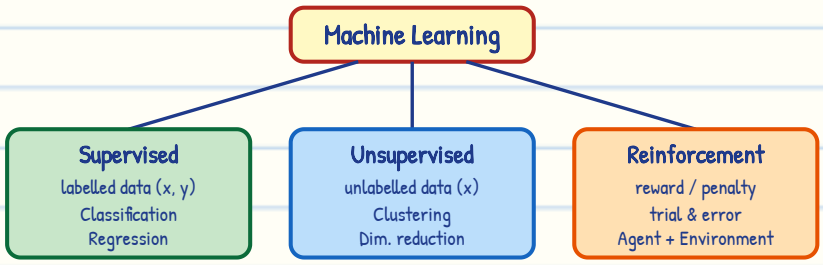


Fig 2: Classification of ML techniques

➤ 2.1 Supervised Learning

- Training data has ***inputs X and known outputs (labels) y***. A "teacher" supplies the correct answer.
- Goal: learn a function ***f: X → y*** that predicts y for new, unseen X.
- Two kinds: ***Classification*** (y = discrete class) and ***Regression*** (y = continuous value).

- Examples: fault / no-fault from vibration; predicting tensile strength from composition.
- Algorithms: Linear/Logistic regression, KNN, Decision tree, Random forest, SVM, Neural nets.

➤ 2.2 Unsupervised Learning

- Data has ***no labels***; the algorithm finds hidden structure by itself.
- Tasks: ***clustering***, association rules, dimensionality reduction (PCA), anomaly detection.
- Examples: grouping machines by operating behaviour; segmenting customers; detecting unusual sensor patterns.

➤ 2.3 Reinforcement Learning (RL)

- An ***agent*** interacts with an ***environment***, takes ***actions***, gets ***rewards***; it learns a ***policy*** that maximises total reward.
- No labelled examples – learning by trial and error. Examples: robot walking, self-driving, HVAC / engine control, game playing.

Point	Supervised	Unsupervised	Reinforcement
Data	Labelled	Unlabelled	No fixed data; feedback
Feedback	Direct (correct answer)	None	Delayed reward
Goal	Predict output	Find structure	Maximise reward
Examples	Spam, price	K-means, PCA	Robot, games

3. Classification Problems

➤ Definition

Classification predicts a ***discrete category (class label)*** for an input. Binary (2 classes: healthy/faulty) or multi-class (normal / unbalance / misalignment / bearing fault).

➤ 3.1 K-Nearest Neighbours (KNN)

- "Lazy" learner – stores all training data. For a new point: find the ***K closest*** training points (Euclidean distance) and take a ***majority vote***.
- Distance: $d = \sqrt{[(x_1-a_1)^2 + (x_2-a_2)^2]}$. Small K → noisy; large K → too smooth. Scale features first!

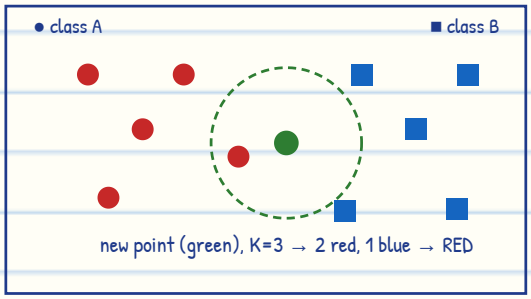


Fig 3: KNN with K = 3

➤ 3.2 Decision Tree

- Tree of if-else questions; each internal node tests a feature, leaves give the class. Easy to interpret (like a flowchart).
- Best split chosen by ***Entropy / Information Gain*** or ***Gini index***.
- Entropy $H(S) = -\Sigma\, p_i \log_2 p_i$; $Gain(S,A) = H(S) - \Sigma\, (|S_v|/|S|)\cdot H(S_v)$; $Gini = 1 - \Sigma\, p_i^2$

Example: 14 samples, 9 "Fault", 5 "OK". $H = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.410 + 0.530 = \mathbf{0.940}$. Pure node → H = 0; 50-50 node → H = 1.

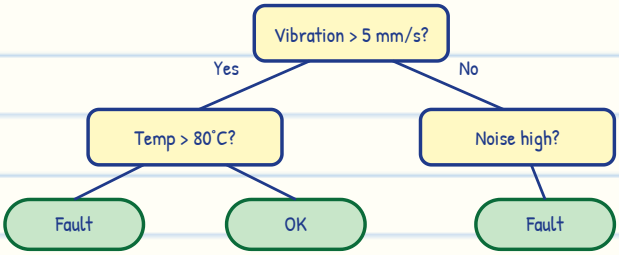


Fig 4: Decision tree for machine health

➤ 3.3 Logistic Regression

- Despite its name, used for ***classification***. Computes $z = w\cdot x + b$ and squashes it with the ***sigmoid***: $\sigma(z) = 1 / (1 + e^{-z})$ giving probability between 0 and 1.
- Rule: if $P(y=1|x) \geq 0.5$ → class 1 else class 0. Loss = binary cross-entropy: $-[y \log p + (1-y) \log(1-p)]$.
- $\sigma(0)=0.5$, $\sigma(2)=0.88$, $\sigma(-2)=0.12$. Decision boundary is a straight line ($z = 0$).

➤ 3.4 Support Vector Machine (SVM) – basics

- Finds the ***hyperplane that separates classes with maximum margin***. Points touching the margin are ***support vectors***.
- Hyperplane: $w\cdot x + b = 0$; margin width = $2/\|w\|$ → maximise margin = minimise $\|w\|$.

- **Kernel trick** (RBF, polynomial) handles non-linear data by mapping to higher dimensions. Parameter C trades margin width vs misclassification.

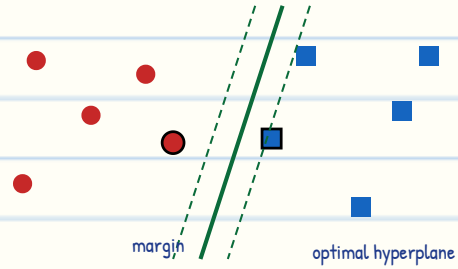


Fig 5: SVM – maximum margin (circled = support vectors)

➤ 3.5 Evaluating Classifiers – Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

- **Accuracy** = (TP+TN)/(TP+TN+FP+FN) · **Precision** = TP/(TP+FP) · **Recall** = TP/(TP+FN)
- **F1** = 2·P·R/(P+R) (harmonic mean). Recall is vital for fault detection – a missed fault (FN) is costly.

Numerical: A fault detector tested on 100 gearboxes: TP = 40, FP = 10, FN = 5, TN = 45. Accuracy = (40+45)/100 = **85%**; Precision = 40/50 = **0.80**; Recall = 40/45 = **0.889**; F1 = 2(0.8)(0.889)/(0.8+0.889) = **0.842**.

4. Regression Problems

➤ Definition

Regression predicts a ***continuous numerical value*** (e.g., tool life in minutes, yield strength in MPa, surface roughness Ra) from input features.

➤ 4.1 Simple Linear Regression

- Model: $\hat{y} = mx + c$ (or $h\theta(x) = \theta_0 + \theta_1 x$). Multiple regression: $\hat{y} = b_0 + b_1x_1 + b_2x_2 + \dots$
- **Least squares:** choose m, c to minimise the sum of squared errors SSE = $\Sigma(y_i - \hat{y}_i)^2$.

- $m = \Sigma(x-\bar{x})(y-\bar{y}) / \Sigma(x-\bar{x})^2$ and $c = \bar{y} - m\cdot\bar{x}$

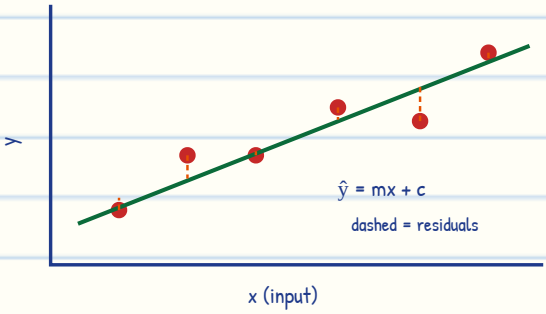


Fig 6: Best-fit line minimises squared residuals

➤ Worked Example

Data: x = 1,2,3,4,5 and y = 2,4,5,4,5. Find the line and predict y at x = 6.

$\bar{x} = 3$, $\bar{y} = 4$. $\Sigma(x-\bar{x})(y-\bar{y}) = (-2)(-2)+(-1)(0)+(0)(1)+(1)(0)+(2)(1) = \mathbf{6}$. $\Sigma(x-\bar{x})^2 = 4+1+0+1+4 = \mathbf{10}$.

$m = 6/10 = \mathbf{0.6}$, $c = 4 - 0.6\times3 = \mathbf{2.2} \rightarrow \hat{y} = \mathbf{0.6x + 2.2}$. At x = 6: $\hat{y} = \mathbf{5.8}$.

Predictions: 2.8, 3.4, 4.0, 4.6, 5.2 → residuals −0.8, 0.6, 1.0, −0.6, −0.2; SSE = 2.4, MSE = 2.4/5 = **0.48**, SST = $\Sigma(y-\bar{y})^2 = 6$, $\mathbf{R^2 = 1 - 2.4/6 = 0.6}$.

➤ 4.2 Cost Function & Gradient Descent

- Cost (MSE): $\mathbf{J(\theta) = (1/2n)\ \Sigma\ (h\theta(x_i) - y_i)^2}$ – measures how wrong the model is.
- ***Gradient descent*** – iterative minimisation: $\theta := \theta - \alpha \cdot \partial J/\partial\theta$ where α = learning rate.
- For linear regression: $\theta_0 := \theta_0 - \alpha(1/n)\Sigma(h-y)$; $\theta_1 := \theta_1 - \alpha(1/n)\Sigma(h-y)x$.
- α too small → slow; α too large → overshoots / diverges. Types: Batch, Stochastic (SGD), Mini-batch.

Mini example: $J(\theta) = \theta^2$, gradient = 2θ, $\alpha = 0.1$, start $\theta = 4$. Step 1: $\theta = 4 - 0.1(8) = \mathbf{3.2}$; step 2: $3.2 - 0.1(6.4) = \mathbf{2.56}$; → heads to 0 (minimum).

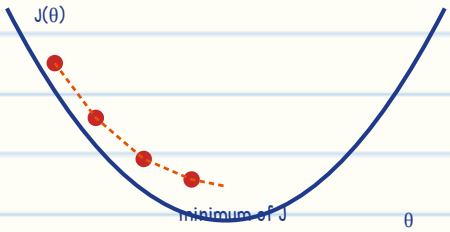


Fig 7: Gradient descent walks downhill

➤ 4.3 Other Regression Metrics

Metric	Formula	Meaning
MAE	$(1/n)\Sigma\lvert y-\hat{y}\rvert$	average absolute error
MSE	$(1/n)\Sigma(y-\hat{y})^2$	penalises big errors
RMSE	$\sqrt{MSE}$	same unit as y
R^2	1 – SSE/SST	fraction of variance explained (1 = perfect)

Other models: polynomial regression (curved data), Ridge/Lasso (regularised), regression trees, SVR, neural nets.

5. Clustering Problems

➤ Definition

Clustering groups ***unlabelled*** data so that points in the same cluster are ***similar*** and those in different clusters are ***dissimilar***. Unsupervised.

- Similarity measured by distance: Euclidean, Manhattan; or cosine similarity.
- Uses: grouping machine-operating regimes, part-family formation in group technology, image segmentation, market segmentation, anomaly detection.

➤ 5.1 K-Means Algorithm

1. Choose K (number of clusters) and pick K initial centroids.
2. ***Assign*** every point to the nearest centroid.
3. ***Update*** each centroid = mean of its points.
4. Repeat 2–3 until centroids stop changing (convergence). Objective: minimise within-cluster sum of squares (WCSS).

Worked example (1-D): points 1, 2, 4, 10, 11, 12; K = 2; initial centroids c_1 = 1, c_2 = 10.

Iteration 1: Cluster 1 = {1,2,4} (4 is 3 away from 1, 6 from 10), Cluster 2 = {10,11,12}. New c_1 = 7/3 = ***2.33***, c_2 = 33/3 = ***11***.

Iteration 2: 4 is 1.67 from 2.33 and 7 from 11 → same clusters → ***converged***. Final: {1,2,4} and {10,11,12}.

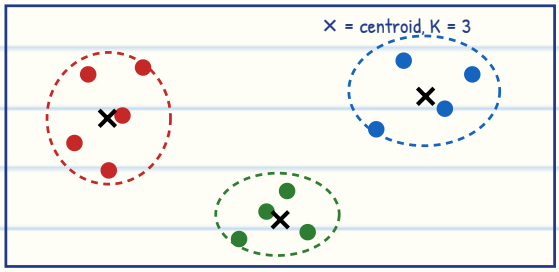


Fig 8: K-means clusters (2-D, K = 3)

- **Choosing K:** Elbow method (plot WCSS vs K; pick the bend), silhouette score.
- **Limitations:** must fix K; sensitive to initial centroids & outliers; finds only round clusters. Fix: K-means++ initialisation.

➤ 5.2 Hierarchical Clustering (basics)

- **Agglomerative (bottom-up):** start with each point as its own cluster, repeatedly merge the two closest clusters until one remains.
- **Divisive (top-down):** start with one big cluster and split.
- Linkage: single (min distance), complete (max), average. Result drawn as a **dendrogram**; cut at a height to get clusters. No need to fix K beforehand.
- Other: DBSCAN (density-based, finds arbitrary shapes, detects noise), Gaussian mixtures. Also PCA for dimensionality reduction.

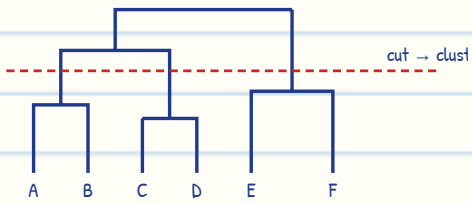


Fig 9: Dendrogram (schematic)

6. Reinforcement Learning

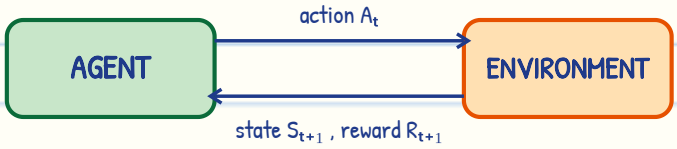


Fig 10: Agent–environment loop

Term	Meaning
Agent	The learner / decision maker (robot, controller)
Environment	Everything the agent interacts with
State (s) / Action (a)	Current situation / choice the agent makes
Reward (r)	Scalar feedback: +ve good, −ve bad
Policy (π)	Mapping state → action (the strategy)
Value / Q-value	Expected total future reward
Discount γ	0–1, importance of future rewards

- ***Exploration vs exploitation:*** try new actions (explore) or use the best known (exploit). ε-greedy: random action with probability ε.
- Modelled as a Markov Decision Process (MDP). Return $G = r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots$

➤ Q-Learning (model-free)

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)\,]$$

Example: $\alpha = 0.5$, $\gamma = 0.9$, all $Q = 0$ initially. Robot moves from s to goal s′, reward $r = 10$, $\max Q(s') = 0 \rightarrow Q(s,a) = 0 + 0.5(10 + 0 - 0) =$ ***5***.

Earlier state s_0 moves into s with $r = 0$: $Q(s_0,a) = 0 + 0.5(0 + 0.9\times5 - 0) =$ ***2.25***. Reward value "propagates backwards" along the path.

- Deep RL (DQN, PPO) uses neural networks for large state spaces. Mechanical uses: robot arm control, gait, drone control, energy management, process parameter tuning.

7. Overfitting, Underfitting & Model Evaluation

- ***Underfitting:*** model too simple – high error on both training and test data (high ***bias***).
- ***Overfitting:*** model memorises training data incl. noise – low training error, high test error (high ***variance***).
- ***Good fit:*** low error on both – generalises well.

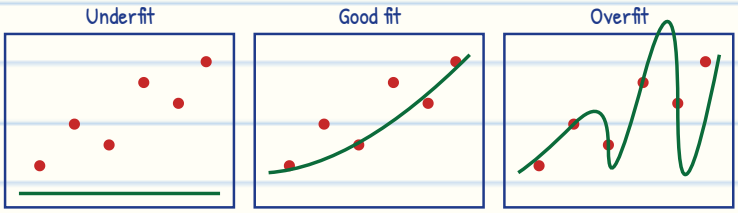


Fig 11: Three fits of the same data

➤ Train / Validation / Test Split

- Split data: ***Train (~70%)*** learns parameters; ***Validation (~15%)*** tunes hyper-parameters; ***Test (~15%)*** final unbiased check – never used in training.
- ***k-fold cross-validation:*** divide into k parts, train on k–1 and test on 1, rotate, average the scores.
- ***Prevent overfitting:*** more data, simpler model, ***regularisation*** (L1/L2), early stopping, dropout, pruning, cross-validation.

➤ Bias–Variance Trade-off

- Total error = ***Bias² + Variance + Irreducible noise***. Increasing complexity lowers bias but raises variance; aim for the sweet spot.

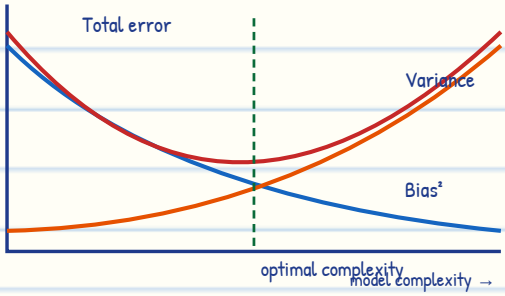


Fig 12: Bias–variance curve

8. Introduction to Neural Networks

- An ***Artificial Neural Network (ANN)*** is inspired by the brain: many simple processing units (neurons) connected by weighted links; learns by adjusting weights.
- Biological: dendrites (inputs) → soma (sum) → axon (output) ↔ Artificial: inputs x_i → weighted sum → activation → output.

➤ 8.1 The Perceptron / Artificial Neuron

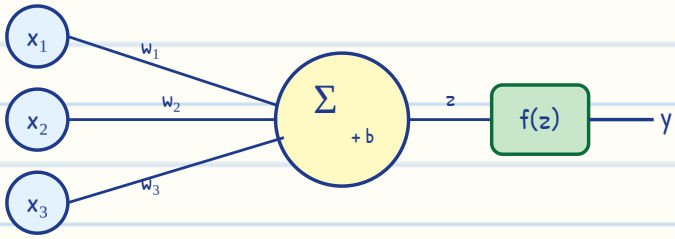


Fig 13: Artificial neuron: $y = f(\sum w_i x_i + b)$

- Weighted sum $z = \Sigma\, w_i x_i + b$; output $y = f(z)$. Weights = importance of inputs; bias b shifts the threshold.
- Perceptron (Rosenblatt 1958): step activation. Update rule: $w_i \leftarrow w_i + \eta\,(target - output)\,x_i$. Solves only ***linearly separable*** problems (AND, OR) – not XOR → need hidden layers.

> ***AND gate:*** $w_1 = w_2 = 1$, $b = -1.5$, step($z \geq 0$). Input (1,1): $z = 0.5 \rightarrow 1$. Input (1,0): $z = -0.5 \rightarrow 0$. ✔

➤ 8.2 Activation Functions

Name	Formula	Range	Use
Step	1 if $z \geq 0$ else 0	{0,1}	perceptron
Sigmoid	$1/(1+e^{-z})$	(0,1)	binary output; vanishing gradient
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	(−1,1)	hidden layers, zero-centred
ReLU	$\max(0, z)$	$[0,\infty)$	default in deep nets, fast
Softmax	$e^{z_i}/\Sigma e^{z_j}$	probabilities sum to 1	multi-class output

➤ 8.3 Multi-Layer Network & Forward Pass

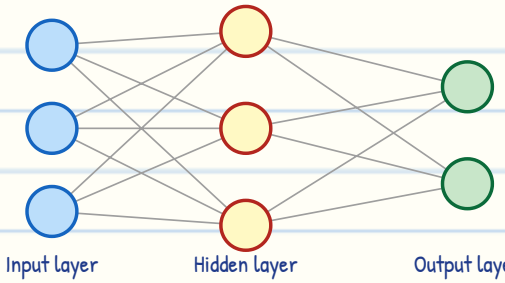


Fig 14: 3–3–2 feed-forward network

Forward pass (one neuron): $x_1 = 1$, $x_2 = 0.5$; $w_1 = 0.4$, $w_2 = 0.6$; $b = -0.1$; sigmoid. $z = 0.4(1) + 0.6(0.5) - 0.1 = \mathbf{0.6} \rightarrow y = 1/(1+e^{-0.6}) = 1/(1+0.549) = \mathbf{0.646}$. This output feeds the next layer.

➤ 8.4 Training – Backpropagation

1. **Forward pass:** compute output $\hat{y}$ from inputs; **loss** $L = \frac{1}{2}(y - \hat{y})^2$ (or cross-entropy).
2. **Backward pass:** using the **chain rule**, compute $\partial L/\partial w$ for every weight, from output layer back to input.
3. **Update:** $w \leftarrow w - \eta\, \partial L/\partial w$ (gradient descent / Adam).
4. Repeat over many **epochs** (one epoch = one pass through all data; batch = samples per update).

Chain rule idea: $\partial L/\partial w = (\partial L/\partial \hat{y})\cdot(\partial \hat{y}/\partial z)\cdot(\partial z/\partial w)$. Error signal flows **backwards** layer by layer.

- **Universal approximation:** a network with one hidden layer and enough neurons can approximate any continuous function.

9. Deep Learning

➤ Definition

Deep Learning = neural networks with **many hidden layers** that automatically learn hierarchical features from raw data (no manual feature engineering). Needs large data and GPUs.

➤ 9.1 Convolutional Neural Network (CNN)

- For **grid-like data – images** (also 2-D spectrograms of vibration signals).
- **Convolution layer:** small filters (kernels, e.g., 3×3) slide over the image to detect edges, textures. Shared weights → fewer parameters.
- **Pooling:** down-samples (max pooling) → reduces size, adds invariance. **ReLU** after convolution. **Fully connected** + softmax at the end.

- Uses: surface-defect detection, weld inspection, crack detection, thermal-image analysis, medical imaging.

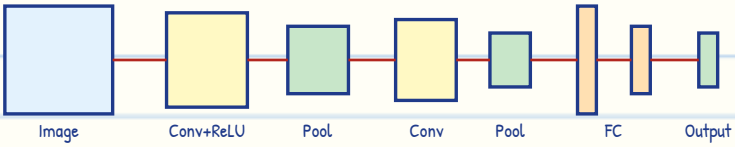


Fig 15: CNN pipeline

➤ 9.2 Recurrent Neural Network (RNN)

- For ***sequential / time-series data*** (speech, text, sensor streams). Has a ***loop***: hidden state $h_t = f(W \cdot x_t + U \cdot h_{t-1} + b)$ carries memory of previous inputs.
- Problem: ***vanishing gradient*** for long sequences → solved by ***LSTM*** and ***GRU*** (gates decide what to remember/forget).
- Uses: remaining-useful-life (RUL) prediction, vibration/time-series forecasting, speech recognition, machine translation.
- Others: Autoencoders (anomaly detection), GANs (synthetic data), ***Transformers*** (attention; LLMs), Physics-Informed Neural Networks (PINNs).

Feature	Classical ML	Deep Learning
Data needed	Small–medium	Very large
Features	Hand-crafted	Learned automatically
Hardware	CPU enough	GPU / TPU
Interpretability	Higher	"Black box"

10. Applications in Mechanical Engineering

Application	ML approach	Input → Output
Predictive maintenance	Regression, LSTM, RUL models	vibration, temperature, current → remaining life
Fault diagnosis (bearings, gears, engines)	SVM, KNN, CNN, decision tree	FFT/spectrogram features → fault class
CFD / FEA surrogate	ANN, Gaussian process, PINN	geometry & boundary conditions → flow/stress field in milliseconds

Material property prediction	Regression, random forest, ANN	composition, heat treatment → strength, hardness
Tool wear / surface finish	ANN, SVR	speed, feed, depth of cut → Ra, wear
Robotics & control	Reinforcement learning	state sensors → motor commands
Defect inspection	CNN	images → defect / no defect
Design optimisation	Surrogate + genetic algorithm	design variables → performance

Case – Predictive maintenance of a pump: sensors give vibration RMS, temperature, pressure. Steps: collect → remove noise/scale → extract features (RMS, kurtosis, FFT peaks) → train classifier (healthy/faulty) or regressor (RUL) → deploy on edge device → alert before failure. Benefit: less downtime, lower cost.

➤ Common Python Tools

- NumPy, Pandas (data) · Matplotlib (plots) · Scikit-learn (classical ML) · TensorFlow / Keras, PyTorch (deep learning).

```
from sklearn.linear_model import LinearRegression
model = LinearRegression().fit(X_train, y_train)
print(model.score(X_test, y_test))  # R² on test data
```

11. Important Exam Questions (Short Q & A)

Q1. Define Machine Learning.

Ans: Learning from experience E for task T with performance P that improves with E (Mitchell). Computers learn from data without explicit programming.

Q2. Differentiate supervised and unsupervised learning.

Ans: Supervised uses labelled data to predict outputs (classification, regression); unsupervised uses unlabelled data to find structure (clustering).

Q3. Classification vs regression?

Ans: Classification output is discrete class; regression output is continuous value.

Q4. What is overfitting? How to avoid?

Ans: Model fits noise, good on train but poor on test. Avoid by more data, regularisation, cross-validation, early stopping, simpler model.

Q5. Write the K-means steps.

Ans: Choose K → initial centroids → assign nearest → recompute means → repeat until no change.

Q6. Define precision, recall, F1.

Ans: Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = 2PR/(P+R).

Q7. What is an activation function? Name four.

Ans: Non-linear function applied to weighted sum giving neuron output: sigmoid, tanh, ReLU, softmax (step).

Q8. Explain backpropagation briefly.

Ans: Error between output and target is propagated backwards using the chain rule to get gradients of the loss w.r.t. weights, which are updated by gradient descent.

Q9. Reinforcement learning – define agent, reward, policy.

Ans: Agent learns by interacting with environment; reward = feedback signal; policy = state-to-action strategy that maximises cumulative reward.

Q10. CNN vs RNN?

Ans: CNN – images/spatial data with convolution & pooling; RNN – sequential data with memory through recurrent hidden state (LSTM/GRU).

➤ Quick Revision

ML = data → model. ***Supervised***: classification + regression. ***Unsupervised***: clustering. **RL**: reward. $m = Sxy/Sxx$; $c = \bar{y} - m\,\bar{x}$. $\theta := \theta - \alpha\nabla J$. $Q \leftarrow Q + \alpha[r + \gamma \max Q' - Q]$. Neuron: $y = f(\Sigma wx + b)$. DL = deep NN (CNN images, RNN sequences).

★ ★ ★ *End of Unit V* ★ ★ ★